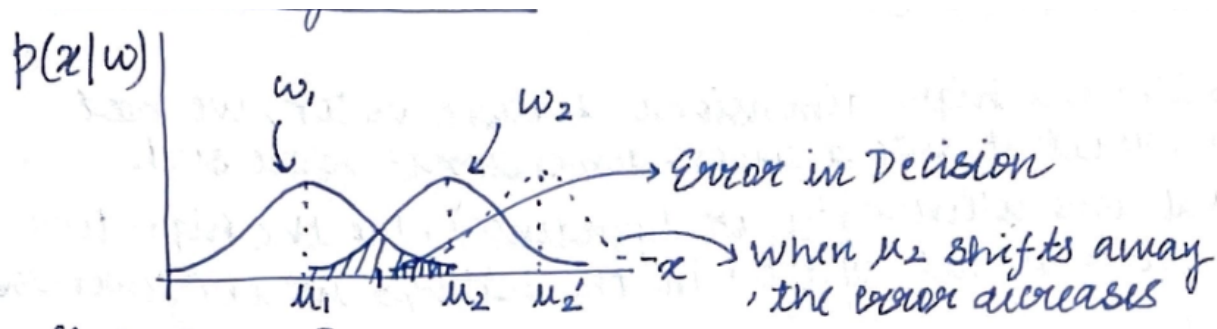


Pattern Recognition & Machine Intelligence

Dimensionality Problem

$$p(x | \omega)$$



Classes: ω_1 & ω_2

$$P(\omega_1) = P(\omega_2); \sigma_1^2 = \sigma_2^2; \mu_1 \neq \mu_2$$

(A priori Probability)

- The discriminating power depends on the distance b/w the means of the features.
- If you find that the discriminating power is less, you may consider adding in more & more # of features! But this leads to the problem of dimensionality.

Why is High Dimensionality a Problem?

Take the discriminating fn:

$$g(x) = -\frac{1}{2}(x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i) - \frac{d}{2} \ln 2\pi - \frac{1}{2} \ln |\Sigma_i| + \ln P(\omega_i)$$

$$\mu_i = \frac{1}{n} \sum_{k=1}^n x_k \Rightarrow O(nd) \quad (d\text{-dim})$$

$$\Sigma_i = \frac{1}{n} \sum_{k=1}^n (x_k - \mu_i)(x_k - \mu_i)^T \Rightarrow O(nd^2)$$

$$\Rightarrow O(cnd^2) \quad (\text{calculating for every class})$$

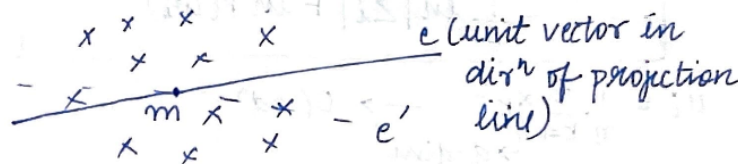
So, we need a way to reduce dimensionality while still maintaining the separability power of the selected features.
→ Given a high-dimensional feature vector, we need to project it into a lower-dimensional space such that any collinearity (or dependence) b/w the high-dim features is not reflected in the features in the lower-dim.
⇒ This means that even if any of the features in the high-dim have any dependence over other features, the features in the low-dim space should all be independent.

Principal Component Analysis (PCA)

Take n # of d -dimensional feature vectors $x_1, x_2, \dots, x_n \in \mathbb{R}^d$.

$$m = \frac{1}{n} \sum_{k=1}^n x_k \quad \text{mean of set of feature vectors}$$

Projection onto 1-dimensional space:



Let e be a unit vector in the direction of projection (a line through m).

Model each x_k as

$$X = m + ae \Rightarrow x_k \approx m + a_k e$$

(representing x_k on a 1-D line, but x_k is d -dimensional) $\rightarrow$ error.

Error fn:

$$J(a_1, \dots, a_n, e) = \sum_{k=1}^n \|(m + a_k e) - x_k\|^2$$

We want to minimize the sum of squared errors.

$$J(a_1, \dots, a_n, e) = \sum a_k^2 \|e\|^2 - 2 \sum a_k e^T (x_k - m) + \sum \|x_k - m\|^2$$

For minimizing: $\frac{\partial J}{\partial a} = 0$

$$\frac{\partial J}{\partial a} = 2 \sum a_k - 2 \sum e^T (x_k - m) = 0 \Rightarrow a_k = e^T (x_k - m)$$

Orthogonal projection of point x_k onto a line in the dir^n of e , passing through the mean m .

$\Rightarrow$ So given a line, the best representation of the feature vectors is the orthogonal projections of the vectors onto the line.

But *which* given line? i.e., which dir^n is best?

Substitute a_k :

$$J(e) = \sum a_k^2 - 2 \sum a_k e^T (x_k - m) + \sum \|x_k - m\|^2 = -e^T \left[\sum (x_k - m)(x_k - m)^T \right] e + \sum \|x_k - m\|^2$$

The second term is independent of e .

Scatter Matrix:

$$S = \sum_{k=1}^n (x_k - m)(x_k - m)^T = n \Sigma$$

Represents the spread of data in space.

$$\Rightarrow J(e) = -e^T S e + \sum \|x_k - m\|^2$$

Independent of e in the last term $\Rightarrow$ minimization of $J(e)$ = maximization of $e^T S e$ (subject to $\|e\| = 1$).

Lagrangian:

$$u = e^T S e - \lambda (e^T e - 1)$$

$$\frac{\partial u}{\partial e} = 2S e - 2\lambda e = 0 \Rightarrow S e = \lambda e \quad (\text{eigenvalue expression})$$

$e \rightarrow$ an eigenvector of S ; $\lambda \rightarrow$ an eigenvalue.

$\rightarrow$ Considering the eigenvectors corresponding to d' # of largest eigenvalues, we take the orthogonal projections of the d -dimensional feature vectors onto our selected eigenvectors.

$\rightarrow$ These projections are called the **Principal Components** and form a dimensionality of d' .

$\rightarrow$ These Principal Components are all orthogonal to each other & are thus uncorrelated.

Scatter Matrix

$$S = \sum (X_k - m)(X_k - m)^T$$

Dimension of X_k : $d \times 1$

$\Rightarrow X_k \otimes X_k^T$: Rank 1 Matrix (Outer Product, always rank 1, given matrices $\neq 0$)

$\Rightarrow S$ is the addition of n rank 1 matrices.

- In order to be invertible, **rank** of S should be d .
- $n > d$ and typically 2 rank-1 matrices when added give 1 rank-2 matrix (generally true).

- S is typically a symmetric and Positive Semidefinite (PSD) matrix. But since Positive Definite (PD), as opposed to PSD, guarantees invertibility, we assume S to be PD.

Positive (Semi) Definite Matrices

Positive Semidefinite

- If the data points are linearly dependent or if the data has fewer data points than features.
- This means that some of its eigenvalues are 0.

$$\boxed{x^T S x \geq 0}, \quad \text{for non-zero vector } x.$$

Positive Definite

- If the data points are linearly independent and there are at least as many data points as features.
- All eigenvalues of a PD matrix are strictly > 0 .

$$\boxed{x^T S x > 0}, \quad \text{for non-zero vector } x.$$

Problem

Prove that the eigenvectors associated with distinct non-zero eigenvalues are orthogonal for a real symmetric matrix.

Solution

Take a real symmetric matrix A ($A^T = A$).

$$\boxed{A\vec{x} = \lambda\vec{x}}$$

Take 2 distinct non-zero values of λ such that:

λ_1 and λ_2 with eigenvectors $\vec{x}$ and $\vec{y}$

$$\boxed{A\vec{x} = \lambda_1\vec{x}} \quad \text{and} \quad \boxed{A\vec{y} = \lambda_2\vec{y}}$$

$$\Rightarrow \vec{y}^T A\vec{x} = \lambda_1 \vec{y}^T \vec{x}$$

$$\Rightarrow (\vec{y}^T A\vec{x})^T = (\lambda_1 \vec{y}^T \vec{x})^T$$

Since A is symmetric, we have:

$$\Rightarrow \boxed{\vec{x}^T A\vec{y} = \lambda_1 \vec{x}^T \vec{y}}$$

Similarly

$$\boxed{\vec{x}^T A\vec{y} = \lambda_2 \vec{x}^T \vec{y}}$$

Equating the two results:

$$\lambda_1 \vec{y}^T \vec{x} = \lambda_2 \vec{x}^T \vec{y}$$

$$\Rightarrow (\vec{x}^T \vec{y})(\lambda_2 - \lambda_1) = 0$$

Since $\lambda_1 \neq \lambda_2$ (distinct eigenvalues):

$$\vec{x}^T \vec{y} = 0$$

$\Rightarrow \vec{x}$ and $\vec{y}$ are orthogonal.

Kernel PCA

- A kernel is defined on a non-empty set X such that:

$$K(x_i, x_j) \rightarrow \mathbb{R}, \quad x_i, x_j \in X$$

where x_i, x_j are feature vectors.

Mercer's Kernel

1. $K(x_i, x_j) = K(x_j, x_i)$ (Symmetric).
2. The Gram Matrix (K_{ij}) will be Positive Definite.

$$K_{ij} = K(x_i, x_j)$$

such that K_{ij} can be represented as $\Phi(x_i)^T \Phi(x_j)$.

Example: $\Phi : X \rightarrow Y$

$$\Phi(x) = [x_1, x_2, \dots, x_d, x_1^2, x_2^2, \dots, x_d^2, \sqrt{2}x_1x_2, \dots], \quad i \neq j$$

$$\text{Space Complexity: } O(d^2) \rightarrow d^2 + 2d$$

$$\Rightarrow \text{Inner Product: } \Phi(x_1)^T \Phi(x_2) \rightarrow O(d^2)$$

Example:

$$X_a = \begin{bmatrix} x_{a1} \\ x_{a2} \end{bmatrix}, \quad X_b = \begin{bmatrix} x_{b1} \\ x_{b2} \end{bmatrix}$$

$$\Phi(x) = [x_1, x_2, x_1^2, x_2^2, \sqrt{2}x_1x_2]$$

$$\begin{aligned} \Rightarrow \Phi(X_a)^T \Phi(X_b) &= [x_{a1}, x_{a2}, x_{a1}^2, x_{a2}^2, \sqrt{2}x_{a1}x_{a2}] [x_{b1}, x_{b2}, x_{b1}^2, x_{b2}^2, \sqrt{2}x_{b1}x_{b2}] \\ &= x_{a1}x_{b1} + x_{a2}x_{b2} + (x_{a1}x_{b1} + x_{a2}x_{b2})^2 \end{aligned}$$

$$\text{Complexity: } O(d) + O(1)$$

$\Rightarrow$ The complexity can be brought down using certain Φ functions.

Linear Kernel: $K(X_1, X_2) = \phi^T(X_1)\phi(X_2) = X_1^T X_2$

Quadratic Kernel: $K(X_1, X_2) = \langle X_1, X_2 \rangle + \langle X_1, X_2 \rangle^2 \rightarrow \phi(x) = \begin{bmatrix} X_1 \\ X_2 \end{bmatrix}$

Polynomial Kernel: $K(X_1, X_2) = (1 + \langle X_1, X_2 \rangle)^M, \quad M \in \mathbb{N}$

Radial Basis Function (RBF) Kernel:

$$K(X_1, X_2) = \exp\left(-\frac{\|x_1 - x_2\|^2}{2\sigma^2}\right) = \exp\left(-\frac{\|x_1\|^2}{2\sigma^2}\right) \cdot \exp\left(-\frac{\|x_2\|^2}{2\sigma^2}\right) \cdot \exp\left(\frac{\langle x_1, x_2 \rangle}{\sigma^2}\right)$$

$\rightarrow$ Taylor Expansion: From finite to infinite dimensional space.

Data: $X_k, \quad k = 1, 2, \dots, l, \quad \sum_{k=1}^l X_k = 0, \quad X_k \in \mathbb{R}^N$

Linear PCA: Orthogonal projection on the eigenvectors of the covariance matrix M_x :

$$C = \frac{1}{l} \sum_{i=1}^l X_i X_i^T$$

$\rightarrow$ We define a non-linear mapping ϕ such that:

$$\phi : \mathbb{R}^N \rightarrow F$$

The dimension of F can be arbitrarily large, but you can perform PCA for certain $\phi(x)$ (even in infinite dimensions).

Assume mapped features as $\phi(x_i)$ are centered at:

$$\sum_{i=1}^l \phi(x_i) = 0$$

$$\Rightarrow \bar{C} = \frac{1}{l} \sum_{i=1}^l \phi(x_i) \phi(x_i)^T$$

Applying PCA for covariance matrix:

$$C\vec{v} = \lambda\vec{v}$$

- Let $F = \text{span}\{\phi(x_1), \dots, \phi(x_l)\}$. If M_x is diagonalizable, then the eigenvectors are in the column space of that M_x .

Eigenvector:

$$\vec{v} = \sum_{i=1}^l \alpha_i \phi(x_i)$$

Substitute into PCA equation:

$$C\vec{v} = \lambda\vec{v}$$

Substitute $\vec{v}$:

$$\phi(x_k) \otimes C\vec{v} = \lambda(\phi(x_k) \otimes \vec{v}) + K \quad (\text{inner product})$$

Expanding the RHS:

$$\begin{aligned} \lambda \begin{bmatrix} \phi(x_1)^T \\ \vdots \\ \phi(x_l)^T \end{bmatrix} \vec{v} &= \lambda \begin{bmatrix} \phi(x_1)^T \\ \vdots \\ \phi(x_l)^T \end{bmatrix} [\alpha_1 \phi(x_1) + \alpha_2 \phi(x_2) + \dots + \alpha_l \phi(x_l)] \\ &= \lambda \begin{bmatrix} \phi(x_1)^T \phi(x_1) & \phi(x_1)^T \phi(x_2) & \dots & \phi(x_1)^T \phi(x_l) \\ \vdots & \vdots & \ddots & \vdots \\ \phi(x_l)^T \phi(x_1) & \phi(x_l)^T \phi(x_2) & \dots & \phi(x_l)^T \phi(x_l) \end{bmatrix} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_l \end{bmatrix} \end{aligned}$$

Define the Gram Matrix:

$$K_{ij} = K(x_i, x_j) = \phi(x_i)^T \phi(x_j) = \lambda K \alpha$$

LHS:

$$\begin{aligned} \frac{1}{l} \begin{bmatrix} \Phi^T(x_1) \\ \vdots \\ \Phi^T(x_l) \end{bmatrix} [\Phi(x_1) \quad \dots \quad \Phi(x_l)] &= \frac{1}{l} \begin{bmatrix} \alpha_1 \\ \alpha_2 \\ \vdots \\ \alpha_l \end{bmatrix} \\ &\Rightarrow \frac{1}{l} K^2 \alpha = \lambda K \alpha \\ &\Rightarrow l \lambda \alpha = K \alpha \quad (\text{Solve for } \alpha). \end{aligned}$$

Test Vector x

$$v^K \Phi(x) = \sum_{i=1}^l \alpha_i^K (\Phi(x_i) \cdot \Phi(x))$$

(Transformed test vector).

Ensure v^K is a unit vector:

$$\begin{aligned} (v^K, v^K) &= 1 \\ \Rightarrow 1 &= \sum_{i,j=1}^l (\alpha_i^K \Phi(x_i)) \cdot (\alpha_j^K \Phi(x_j)) \\ &= \sum_{i,j=1}^l \alpha_i^K \alpha_j^K (\Phi(x_i) \cdot \Phi(x_j)) \\ &\Rightarrow (\alpha^K)^T K \alpha^K = 1 \end{aligned}$$

Note: PCA does not use class labels.

Linear Discriminant Analysis (LDA)

- Maximizes class separability while reducing dimensionality.
- **Advantage over PCA:** LDA takes class labels into account.

Take n d -dimensional vectors $x_1, x_2, \dots, x_n$, with $x_i \in \mathbb{R}^d$. Datasets: $D_1 \rightarrow \omega_1, \quad D_2 \rightarrow \omega_2$ (classes).

Linear Discriminant Function

$$y = W^T X$$

where X is $d \times 1$, W is $d \times 1$, and y is scalar (1×1).

Question: Given two lines, which should be chosen as the LDA projection line?

Answer:

1. Maximize the distance between the sample means.
2. Minimize the projected variance (scatter) within each class.

Sample Means (Projected)

Sample mean for class ω_i :

$$m_i = \frac{1}{n_i} \sum_{x \in D_i} X$$

Projected sample mean:

$$\tilde{m}_i = \frac{1}{n_i} \sum_{x \in D_i} W^T X = W^T m_i$$

Projected Intra/Within-Class Scatter

$$\begin{aligned} \tilde{s}_i^2 &= \sum_{y \in \omega_i} (y - \tilde{m}_i)^2 = \sum_{x \in D_i} (W^T (x - m_i))^2 = \sum_{x \in D_i} W^T (x - m_i) (x - m_i)^T W \\ &\Rightarrow \boxed{\tilde{S}_i = W^T S_i W} \end{aligned}$$

where S_i is the within-class scatter matrix for class ω_i .

Total Within-Class Projected Scatter

$$S_W = \sum_{i=0}^d \tilde{S}_i^2$$

For $D_1 \rightarrow \omega_1$ and $D_2 \rightarrow \omega_2$:

$$\tilde{s}^2 = \tilde{s}_1^2 + \tilde{s}_2^2 = W^T S_W W$$

where S_W is the total within-class scatter.

Between-Class Scatter

$$\|\tilde{m}_1 - \tilde{m}_2\|^2 = |W^T (m_1 - m_2)|^2 = W^T (m_1 - m_2) (m_1 - m_2)^T W = W^T S_B W$$

where S_B is the between-class scatter matrix.

LDA Optimization

We want:

$$W^* = \operatorname{argmax}_W J(W) = \operatorname{argmax}_W \frac{\|\tilde{m}_1 - \tilde{m}_2\|^2}{\tilde{s}_1^2 + \tilde{s}_2^2} = \operatorname{argmax}_W \frac{W^T S_B W}{W^T S_W W}$$

This is a **generalized Rayleigh quotient**.

Rayleigh Quotient

For a matrix A and vector x :

$$\frac{x^T A x}{x^T x} \text{ is scalar } (= k)$$

Maximizing this quotient yields the eigenvector corresponding to the maximum eigenvalue of A .

$$\Rightarrow x^T A x = K x^T x$$

$$\Rightarrow \boxed{Ax = Kx} \rightarrow \text{Eigenvector}$$

$$\Rightarrow \boxed{\max\{K\} = \max\{\text{eigenvalues}\}}$$

Generalized Rayleigh Quotient

$$\frac{x^T A x}{x^T B x}$$

Cholesky Decomposition: $B = LL^T \rightarrow$ Lower Traiangular Matrix Decomposition
- B should be PSD: A necessary condition for Cholesky decomposition.

Proof:

$$z^T B z = z^T L L^T z = (L^T z)^T (L^T z) = \|L^T z\|^2 \geq 0 \Rightarrow PSD.$$

$$\begin{aligned} \rightarrow \frac{x^T A x}{x^T B x} &= \frac{x^T A x}{x^T L L^T x} = \frac{x^T L L^{-1} A L^{-T} L^T x}{x^T L L^T x} \\ &\Rightarrow \frac{x^T A x}{x^T B x} = \frac{z^T L^{-1} A L^{-T} z}{z^T z} \end{aligned}$$

$\rightarrow$ The maximum value is the largest eigenvalue of $L^{-1} A L^{-T}$.

$$\begin{aligned} &\rightarrow L^{-1} A L^{-T} z = \lambda z \\ &\Rightarrow L^{-1} A L^{-T} L^T x = \lambda L^T x \\ &\Rightarrow L^{-1} A x = \lambda L^T x \\ &\Rightarrow \boxed{A x = \lambda B x} \end{aligned}$$

Applying to LDA:

$$\begin{aligned} &\boxed{S_B W = \lambda S_W W} \\ &(m_1 - m_2)(m_1 - m_2)^T W = \lambda S_W W \\ &C(m_1 - m_2) = \lambda S_W W \\ &\boxed{C' S_W^{-1} (m_1 - m_2) = W} \end{aligned}$$

Multi-class LDA (MDA)

C classes $\rightarrow (C - 1)$ dimensional space projection

$$x_1, x_2, \dots, x_n; \quad \text{where } x_i \in \mathbb{R}^d$$

Total Within-Class Scatter

$$S_W = \sum_{i=1}^C S_i = \sum_{i=1}^C \sum_{x \in D_i} (x - m_i)(x - m_i)^T$$

where

$$\boxed{m_i = \frac{1}{n_i} \sum_{x \in D_i} x} \quad \Rightarrow \quad \boxed{m = \frac{1}{n} \sum_{x \in D} x = \frac{1}{n} \sum_{i=1}^C \sum_{x \in D_i} x = \frac{1}{n} \sum_{i=1}^C n_i m_i}$$

with $D = D_1 \cup D_2 \cup \dots \cup D_C$.

Total Sample Mean

$$\begin{aligned} S_T &= \sum_{x \in D} (x - m)(x - m)^T \\ &= \sum_{i=1}^C \sum_{x \in D_i} \underbrace{(x - m_i)}_a + \underbrace{(m_i - m)}_b \underbrace{(x - m_i)}_a + \underbrace{(m_i - m)}_b^T \end{aligned}$$

$$\begin{aligned}
&= \sum_{i=1}^C \sum_{x \in D_i} (x - m_i)(x - m_i)^T + (m_i - m)(m_i - m)^T \\
&\quad + (x - m_i)(m_i - m)^T + (m_i - m)(x - m_i)^T \\
&= S_W + \sum_{i=1}^C n_i (m_i - m)(m_i - m)^T + \sum_{i=1}^C \left\{ \sum_{x \in D_i} (x - m_i) \right\} (m_i - m)^T \\
&\quad + \sum_{i=1}^C (m_i - m) \underbrace{\sum_{x \in D_i} (x - m_i)^T}_0
\end{aligned}$$

Noting that $\sum_{x \in D_i} (x - m_i) = 0$, the cross terms vanish.

Total Scatter

$$\boxed{S_T = S_W + S_B}$$

where S_B is the total between-class scatter.

Note: S_T is independent of processing if you remain in the feature space. Thus, S_T remains fixed for:

$$S_W \uparrow, S_B \downarrow \quad \text{or} \quad S_W \downarrow, S_B \uparrow$$

MDA Projection

Projecting $X_i \rightarrow y_i$:

$$y = W^T X$$

where $W \in \mathbb{R}^{d \times (C-1)}$, $y \in \mathbb{R}^{C-1}$.

Projected sample mean for class ω_i :

$$\bar{m}_i = \frac{1}{n_i} \sum_{y \in y_i} y = \frac{1}{n_i} \sum_{x \in D_i} W^T x = W^T m_i$$

(not a vector but an $(C-1)$ -dimensional mean).

Projected total mean:

$$\begin{aligned}
\bar{m} &= \frac{1}{n} \sum_{y \in y} y = \frac{1}{n} \sum_{i=1}^C \sum_{x \in D_i} W^T x_i = \frac{1}{n} \sum_{i=1}^C n_i W^T m_i \\
&\Rightarrow \boxed{\bar{m} = \frac{1}{n} \sum_{i=1}^C n_i \bar{m}_i}
\end{aligned}$$

Projected within-class scatter:

$$\bar{S}_W = \sum_{i=1}^C \sum_{y \in y_i} (y - \bar{m}_i)(y - \bar{m}_i)^T = W^T S_W W$$

Projected between-class scatter:

$$\bar{S}_B = \sum_{i=1}^C n_i (\bar{m}_i - \bar{m})(\bar{m}_i - \bar{m})^T = W^T S_B W$$

Optimization Criterion

$$\boxed{J(W) = \frac{|W^T S_B W|}{|W^T S_W W|}}$$

Here, $J(W)$ is not a scalar anymore but a ratio of determinants.

No closed form solution — take each column of W separately:

$$\boxed{S_B w_i = c S_W w_i}$$

Linear Discriminant Function

$$g(x) = W^T x + W_0$$

where $W \in \mathbb{R}^d$ is the weight vector and $W_0 \in \mathbb{R}$ is the bias.
Define W such that:

$$g(x) > 0 \Rightarrow x \in \omega_1, \quad g(x) < 0 \Rightarrow x \in \omega_2, \quad g(x) = 0 \Rightarrow \text{Decision boundary.}$$

We can write:

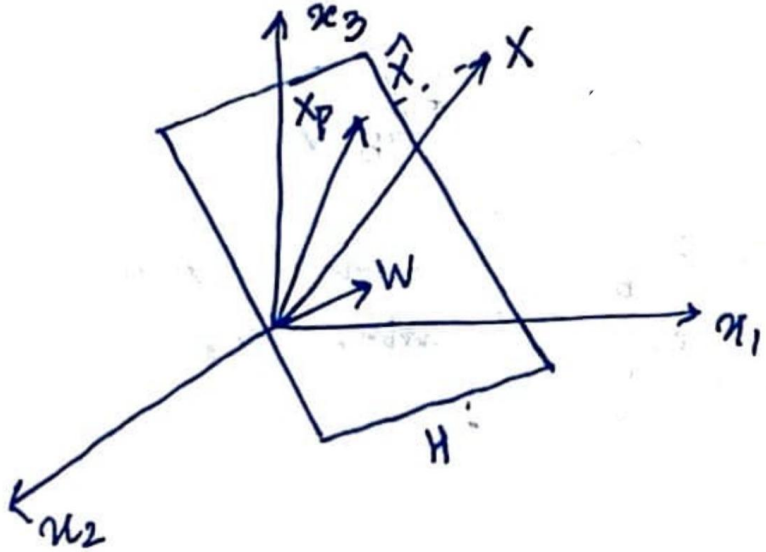
$$g(x) = W'^T x', \quad W' = \begin{bmatrix} W \\ W_0 \end{bmatrix}, \quad x' = \begin{bmatrix} x \\ 1 \end{bmatrix}$$

Suppose we have two vectors x_1 and x_2 on the decision boundary:

$$g(x_1) = g(x_2) = 0$$

$$\Rightarrow \boxed{W^T(x_1 - x_2) = 0}$$

Thus W is orthogonal to the hyperplane H . Let $\hat{x}$ be orthogonal to x_p and parallel to W :



$$x_p + \hat{x} = x, \quad \hat{x} \parallel W$$

The unit vector in the direction of W is:

$$\frac{W}{\|W\|}$$

Distance from the Hyperplane

Let $\hat{x} = r \frac{W}{\|W\|}$, then:

$$g(x) = W^T x + W_0 = W^T(x_p + \hat{x}) + W_0$$

Since $g(x_p) = W^T x_p + W_0 = 0$:

$$g(x) - g(x_p) = r \frac{W^T W}{\|W\|} \Rightarrow g(x) = r \|W\|$$

Thus:

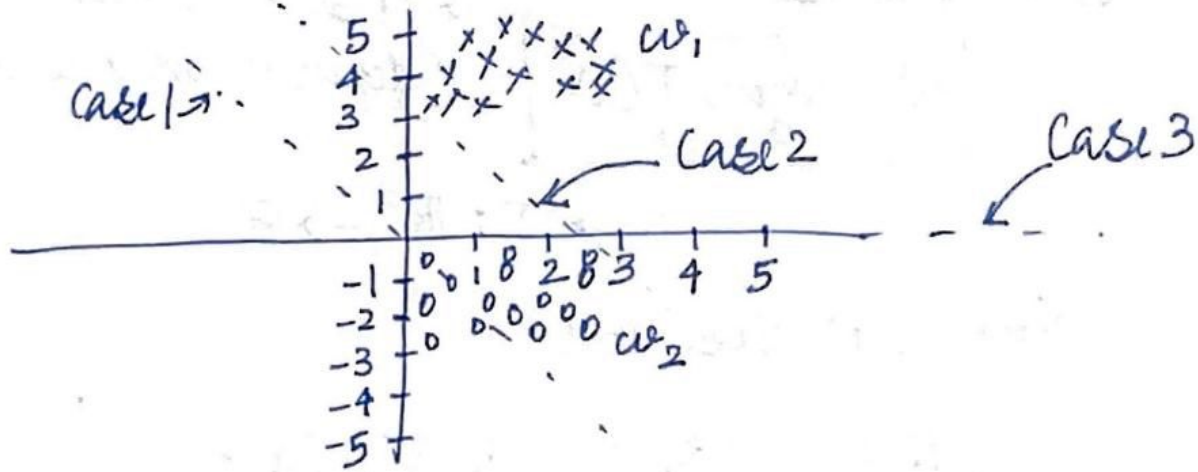
$$\boxed{r = \frac{g(x)}{\|W\|}}$$

If x is the origin:

$$\boxed{r_0 = \frac{W_0}{\|W\|}}$$

The direction of W is considered to be the direction of the plane.

Examples



Case 1: $W = \begin{bmatrix} 1 \\ 1 \end{bmatrix}$, $W_0 = 0 \Rightarrow g(x) > 0$ for $x \in \omega_1$, $g(x) < 0$ for $x \in \omega_2$.

Case 2: Change $W_0 \rightarrow W_0 < 0$

Case 3: $W_0 = 0$, $W = \begin{bmatrix} 1 \\ 0 \end{bmatrix}$

Case 4: $W = \begin{bmatrix} 0 \\ 1 \end{bmatrix}$, $W_0 = -2$

Multiclass Classification

1. C classes: Select any two classes and calculate the decision boundary.
2. Using discriminant functions:

$$g_i(x), \quad i = 1, 2, \dots, C$$

Assign x to ω_j if:

$$g_j(x) > g_i(x), \quad \forall i \neq j$$

Loss Minimization using Gradient Descent

An iterative process to minimize the cost function $J(a)$:

$$a^* = \underset{a}{\operatorname{argmin}} J(a), \quad J: \mathbb{R}^d \rightarrow \mathbb{R}$$

Initialize $a(0)$ close to 0 and update:

$$a(k+1) = a(k) - \eta(k) \nabla J(a(k))$$

where $\eta(k)$ is the learning rate (> 0). We do not simply solve $\nabla J(a) = 0$ because it may be a transcendental equation without a closed-form solution.

Discriminant Function

For every class, we define a function $g_i(x)$, and for whichever class i it turns out to be the greatest, x is classified to that corresponding class i :

$$g_i(x) = p(\omega_i | x) := p(x | \omega_i) \cdot p(\omega_i)$$

(Bayes Minimum Error Classifier) or

$$g_i(x) = -R(\alpha_i | x)$$

(Bayes Minimum Risk Classifier)

Discriminant Functions based on Bayes Minimum Error Classifier

For C classes, $p(\omega_i | x)$ has the common term $\sum_j p(\omega_j) p(x | \omega_j)$, so we write:

$$p(\omega_i | x) = p(x | \omega_i) \cdot p(\omega_i)$$

Let:

$$g_i(x) = f(p(\omega_i | x))$$

If f is monotonically increasing, then $g_i(x)$ can be used as a discriminating function, since $f(p(\omega_i | x))$ is maximized when $g_i(x)$ is maximized.

→ A very convenient choice for f is $\ln$, giving:

$$g_i(x) = \ln p(x | \omega_i) + \ln p(\omega_i)$$

Question. Why discriminant functions?

Answer. We want to be able to create physical decision boundaries instead of having to calculate so many probabilities.

Discriminant Function under Multivariate Normal Distribution

If our feature vector consists of features from multiple varying attributes such as shape, color, intensity, etc., we often use a multivariate Gaussian distribution.

Let x be d -dimensional. The density is:

$$p(x) = \frac{1}{(2\pi)^{d/2} |\Sigma|^{1/2}} \exp \left[-\frac{1}{2} (x - \mu)^T \Sigma^{-1} (x - \mu) \right]$$

Thus:

$$p(x | \omega_i) = \frac{1}{(2\pi)^{d/2} |\Sigma_i|^{1/2}} \exp \left[-\frac{1}{2} (x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i) \right]$$

Taking logs:

$$g_i(x) = -\frac{d}{2} \ln 2\pi - \frac{1}{2} \ln |\Sigma_i| - \frac{1}{2} (x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i) + \ln p(\omega_i)$$

The first term is class-independent and can be ignored, giving:

$$g_i(x) = \ln p(\omega_i) - \frac{1}{2} [\ln |\Sigma_i| + (x - \mu_i)^T \Sigma_i^{-1} (x - \mu_i)]$$

Special cases:

- If all components of x are statistically independent, Σ_i will be diagonal.
- If every component has the same variance σ^2 , then $\Sigma_i = \sigma^2 I$.

Regularization in Discriminant Functions

If W is scaled by a factor λ , then:

$$S_j - S_{y_i} = W_j^T x_i - W_{y_i}^T x_i$$

also scales by λ . Some terms might be $> \Delta$, others might not, so we need to choose a version of W consistently.

We add a regularization term:

$$R(W) = \lambda \sum_k \sum_\ell W_{k\ell}^2$$

Total loss:

$$L = \frac{1}{N} \sum_i L_i + R(W)$$

For example:

$$L = \frac{1}{N} \sum_i \sum_{j \neq y_i} \max \{0, f(x_i, W)_j - f(x_i, W)_{y_i} + \Delta\} + \lambda \sum_k \sum_\ell W_{k\ell}^2$$

Here Δ and λ are hyperparameters controlling the trade-off; Δ is often set to 1, and λ is chosen during minimization. On visualization, the loss L is found to be convex and can be minimized using convex optimization techniques.

Optimization (Gradient Descent)

$$\nabla_{W_{y_i}} = -\frac{1}{N} \sum_i \sum_{j \neq y_i} [x_i (W_j^T x_i - W_{y_i}^T x_i + \Delta > 0)] + \eta W_{y_i}$$
$$\nabla_{W_j} = \frac{1}{N} \sum_i \sum_{j \neq y_i} [x_i (W_j^T x_i - W_{y_i}^T x_i + \Delta > 0)] + \epsilon_j W_j$$

Update rules:

$$W_{y_i} := (1 - \eta)W_{y_i} + \frac{1}{N} \sum_i \sum_{j \neq y_i} [x_i (W_j^T x_i - W_{y_i}^T x_i + \Delta > 0)]$$
$$W_j := W_j - \nabla_{W_j}$$

Q: What if the loss function is not convex?

A: In such cases, a simple gradient descent optimizer might not always give the global minimum. We can use other optimizers such as **Stochastic Gradient Descent**, **Mini-Batch Gradient Descent**, or momentum-based algorithms such as **NAdam**.

Batch, Stochastic, and Mini-Batch Gradient Descent

Batch Gradient Descent (BGD)

Calculate loss using all training samples, then update the weights using the gradients of the loss.

Advantage: Less frequent updates → less computation required.

Disadvantage: Might converge to local minima.

$$L = \sum_{i=1}^N (F(x_i) - y_i)^2$$
$$W_j := W_j - \gamma \frac{\partial L}{\partial W_j}$$

Stochastic Gradient Descent (SGD)

Calculate loss for each sample and update weights before moving to the next sample.

Advantages:

1. Frequent updates → faster learning.
2. Noisy updates help avoid local minima.

Disadvantages:

1. High computational cost.
2. Noisy gradient signal can cause parameter updates to jump around.

$$L_i = (f(x_i) - y_i)^2$$
$$W_j := W_j - \gamma \frac{\partial L_i}{\partial W_j}$$

Mini-Batch Gradient Descent

Divide the training set into batches of size b , calculate loss for each batch, and update weights before moving to the next batch.

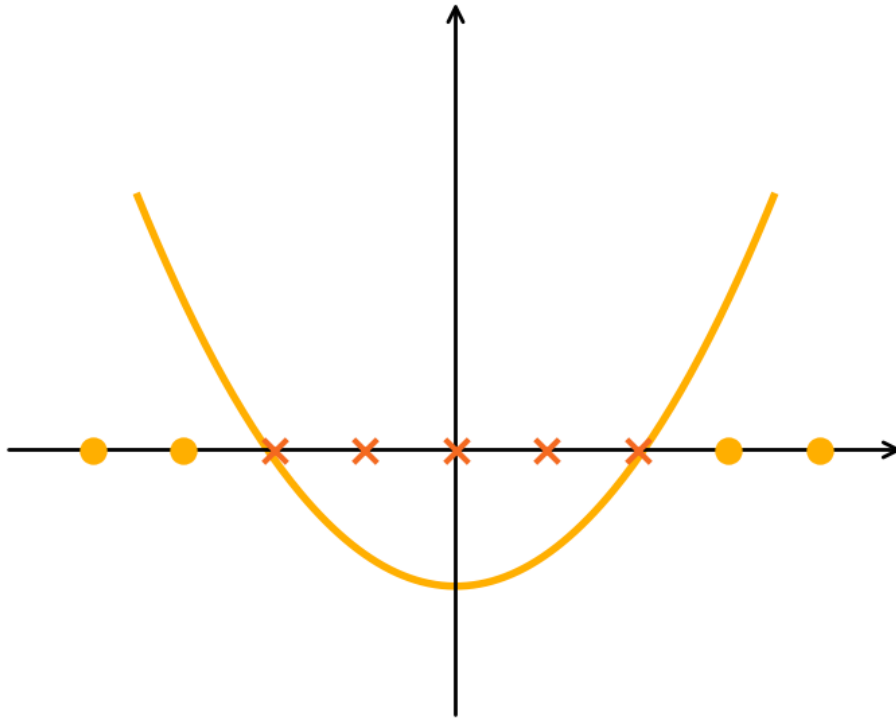
$$\#iterations = \#batches, \quad \#epochs = \#times \text{ to scan entire data}$$

Its advantages and drawbacks are a mix of those of BGD and SGD.

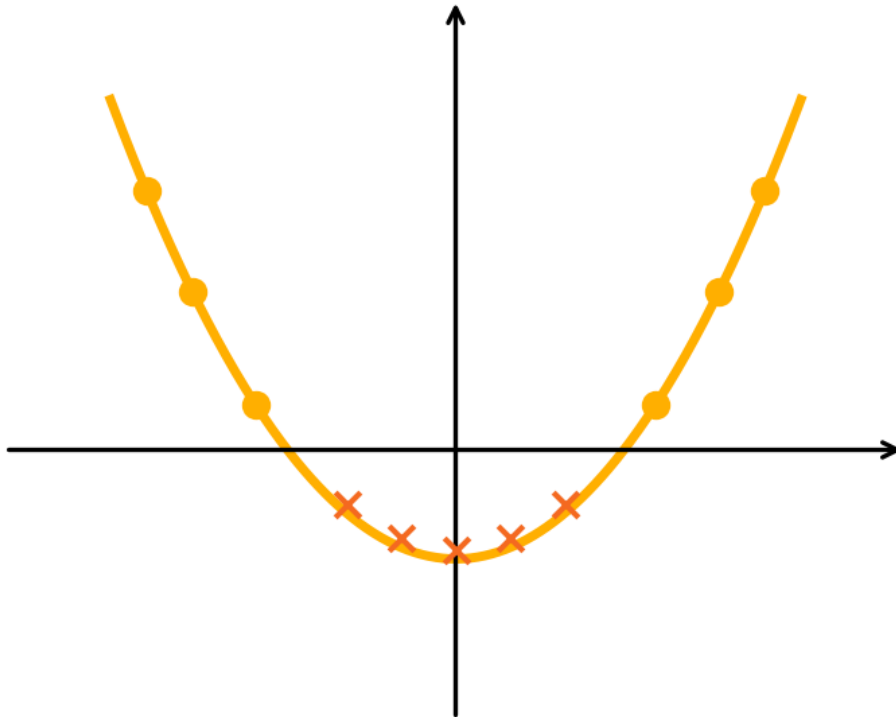
Nonlinearity

– What happens if in the given dimensional space, the set of classes are not linearly separable?

Eg:



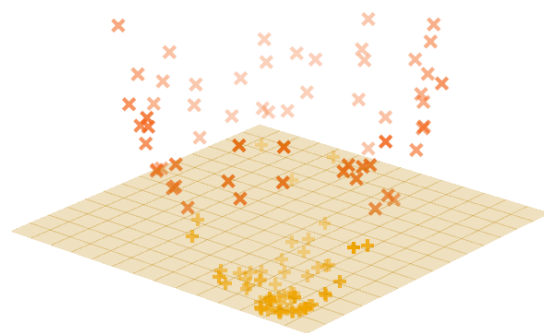
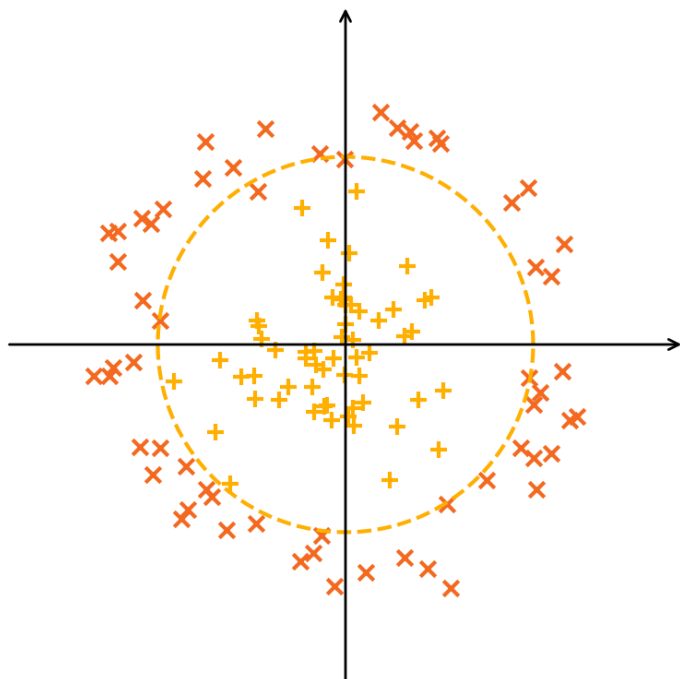
- One approach would be to just use a **Non-Linear** separator itself, as shown, above.



- The other approach is to use a non-linear mapping of higher dimension and then use a linear separator.

Eg:

$$z = x_1^2 + x_2^2$$



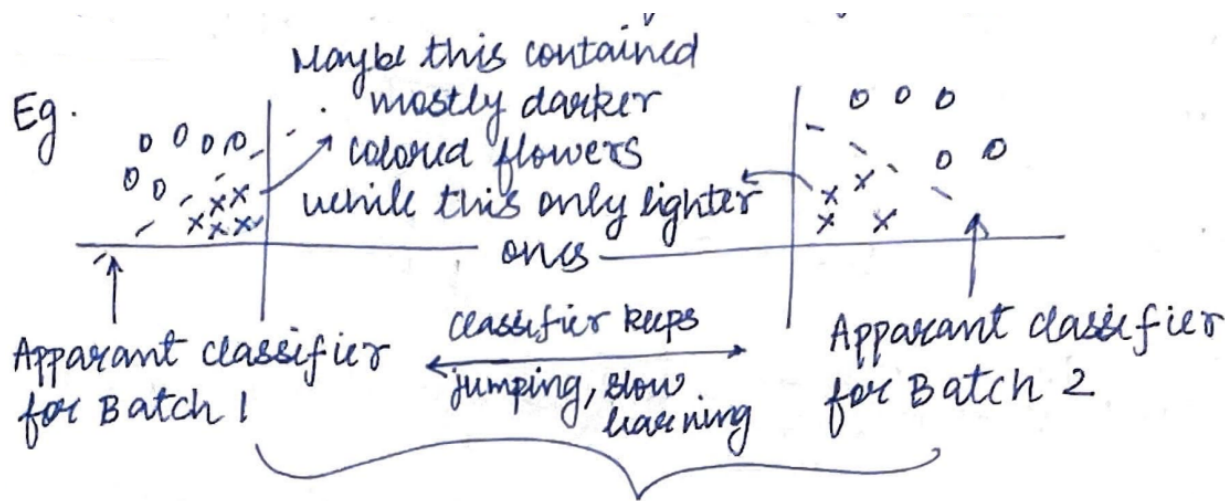
Linear separation through plane

– Neural Networks use such non-linear mappings or functions to convert linearly non-separable feature vectors into linearly separable ones.

Batch Normalization

For most of practical applications, data is fed in batches for training a model. It is entirely possible that the distribution of feature vectors comes out to be different for different batches.

Eg:



Covariate Shift

Could happen due to a reason as small as color difference in the classes of 2 batches.

- Now even after performing normalization, there's no guarantee that during updation of the weights after every epoch, our weights still stay within scale & do not explode or vanish.
- So we actually need to apply normalization to the outputs of the activation functions at each layer too.
- But now if you perform this type of normalization after every epoch, you will find that eventually all your data loses its identity & all the data will now have the same distribution.
- So you do the following re-parameterization:

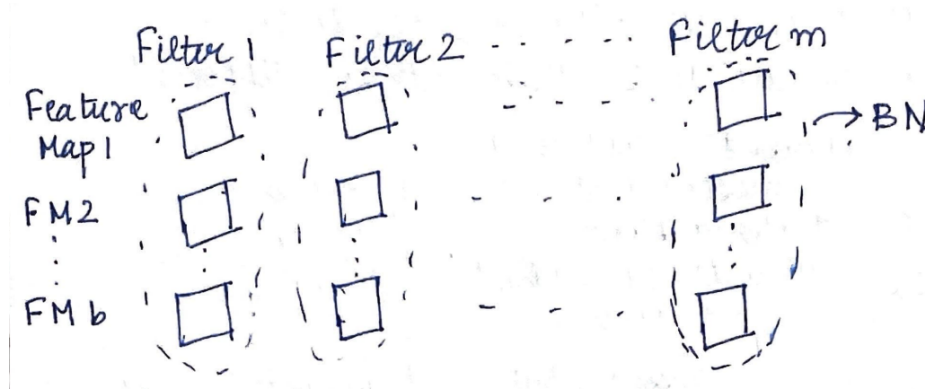
$$n \rightarrow \frac{x - \mu}{\sqrt{\sigma^2 + \epsilon}}$$

Normalization: $x \rightarrow g \cdot x + b$

Here, g and b are **trainable parameters** (like weights).

Visualization of Normalization

Take a convolutional layer with m filters & batch size b . Output of conv. layer:



Batch Norm works for each filter separately using all outputs of that filter. → Does not allow normalization till the entire batch is processed.

Layer Normalization – Works for each input separately using outputs of all filters for that input. → Useful for processing sequential data such as in LSTMs.

Regularization: L1 & L2

- Helps avoid overfitting

$$\text{Cost Function} = \text{Loss} + \text{Regularization Term}$$

$$\text{L1: } \lambda \sum |w_i|$$

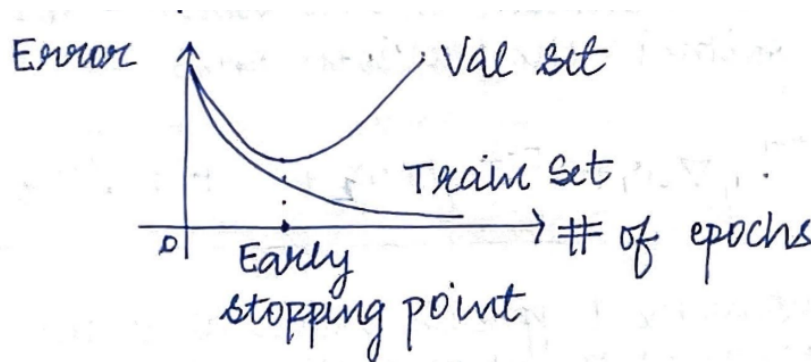
- Can make some weights = 0, so helps with model compression.

$$\text{L2: } \lambda \sum \|w_i\|^2$$

- *Weight Decay*: Pushes larger weights closer to 0.

Early Stopping

- Automatically stops training when the optimal number of epochs have been completed.



Algorithm:

1. Split data into train, val & test sets.
2. After every epoch:
 - (a) Evaluate model performance on val set.
 - (b) Save the best model evaluated on val set.
3. Use final model that has the best val set performance.

Momentum based Gradient Descent

If the last few updates have pointed you in the same direction, then maybe it makes sense to take larger steps in that direction.

$$\text{update}_0 = 0$$

$$\text{update}_1 = \gamma \cdot \text{update}_0 + \eta \nabla w_1 = \eta \nabla w_1$$

$$\text{update}_2 = \gamma \cdot \text{update}_1 + \eta \nabla w_2 = \gamma \eta \nabla w_1 + \eta \nabla w_2$$

A fraction of the direction pointed earlier + current direction.

→ Your faith lies maximum in the current direction and then decaying faith in the previous ones:

$$\text{update}_t = \gamma^{t-1} \eta \nabla w_1 + \gamma^{t-2} \eta \nabla w_2 + \dots + \eta \nabla w_t$$

→ But the momentum-based approach allows GD to take larger steps even when the slope is shallow, and in some cases it might make it oscillate around the minima. This way GD still converges faster than vanilla GD, but it also shows there's still scope for improvement.

Nesterov Accelerated Gradient Descent

Look before you leap:

$$W_{t+1} = W_t - \text{update}_t$$

$$\text{update}_t = \gamma \cdot \text{update}_{t-1} + \eta \nabla W_t$$

(Update rule for MGD)

- Instead of calculating the gradient first and moving one big step of $\gamma \text{update}_{t-1} + \eta \nabla W_t$, move a smaller step of $\gamma \text{update}_{t-1}$ and then calculate the gradient:

$$W_{\text{look-ahead}} = W_t - \gamma \cdot \text{update}_{t-1}$$

$$\text{update}_t = \gamma \cdot \text{update}_{t-1} + \eta \nabla W_{\text{look-ahead}}$$

$$W_{t+1} = W_t - \text{update}_t$$

Similar update rule for bias as well.

- Basically, when you know you're going to move in the direction of the history, move there first and then calculate the gradient for the larger step.

→ This results in smaller oscillations around the minima.

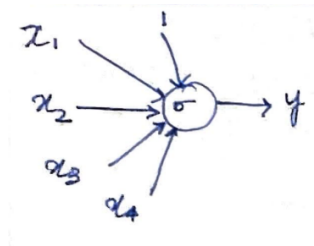
→ But the larger step taken by the gradient can still be optimized if we use an optimal or adaptive learning rate.

Line Search

- Take k different values for η and at each gradient calculation, use the η which gives the lowest loss.

Disadvantage: Computation $\rightarrow \times k$

Gradient Descent with Adaptive Learning Rate



$$y = f(x) = \frac{1}{1 + \exp[-(WX + b)]}$$

$$X = \{x_1, x_2, x_3, x_4\}, \quad W = \{w_1, w_2, w_3, w_4\}$$

- Suppose that x_2 is very sparse (≈ 0 for most inputs), then $\nabla w_2 \approx 0$ for most inputs and w_2 won't get enough updates.

- If x_2 is not important, no problem. But if it is important, we want w_2 to be learned properly.

- So we can change the learning rate specifically for x_2 so that it gets updated faster compared to other features.
- *Decay the learning rate for parameters in proportion to their update history:

$$\text{Learning Rate} \propto \frac{1}{\text{Update History}}$$

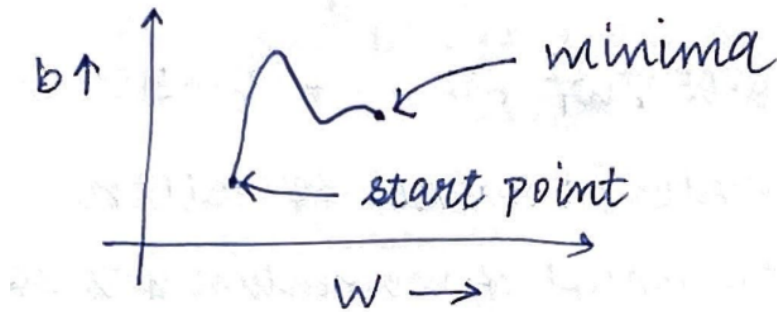
For a sparse feature, the update history will be small; for a dense feature, it will be large.

Adagrad

$$\begin{aligned} V_t &= V_{t-1} + (\nabla W_t)^2 && \rightarrow \text{History Vector} \\ W_{t+1} &= W_t - \frac{\eta}{\sqrt{V_t + \epsilon}} \cdot \nabla W_t \end{aligned}$$

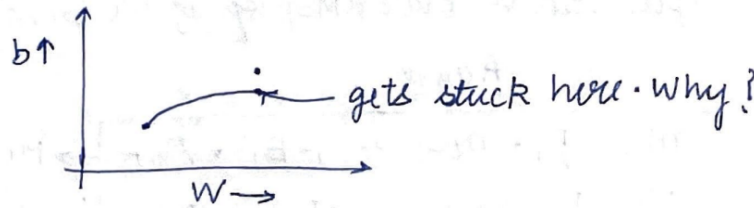
Q: How does the above approach help in practice?

- Take a sparse feature vector x : In the initial few steps, we find that all 3 of GD, MGD & NAG move in the dir^n of b , because w undergoes very few updates due to x being sparse.



(minima and start point illustration)

Adagrad:



(gets stuck here - Why?)

- Adagrad ensures that a weight vector getting updated very frequently should have its LR decayed just as quickly due to the growing denominators.
- So over time, the LR for b decays so much that there will be no further updates to b .
- This is improved upon by RMSProp.

RMSProp

$$\begin{aligned} V_t &= \beta \cdot V_{t-1} + (1 - \beta)(\nabla W_t)^2 && \rightarrow \text{History Vector} \\ W_{t+1} &= W_t - \frac{\eta}{\sqrt{V_t + \epsilon}} \cdot \nabla W_t \end{aligned}$$

where $\beta \geq 0.95$.

- Dependence or faith in history keeps decaying:

$$\begin{aligned} V_1 &= 0.05 \nabla W_1^2 \\ V_2 &= 0.95 \times 0.05 \nabla W_1^2 + 0.05 \nabla W_2^2 \\ &\vdots \\ V_n &= (0.95)^{n-1} \times 0.05 \nabla W_1^2 + \dots + 0.05 \nabla W_n^2 \end{aligned}$$

Ancient history: Almost forgotten.

– But we see that the concept of momentum gets lost in all these optimizations, and it can actually be an added optimization over RMSProp if we bring it back.

Adam

$m_t = \beta_1 \cdot m_{t-1} + (1 - \beta_1) \cdot \nabla W_t \quad \rightarrow \text{Moving avg of grads (momentum)}$ $V_t = \beta_2 \cdot V_{t-1} + (1 - \beta_2) \cdot (\nabla W_t)^2$ $\hat{m}_t = \frac{m_t}{1 - \beta_1^t} \quad \rightarrow \text{Bias correction term}$ $\hat{V}_t = \frac{V_t}{1 - \beta_2^t}$ $W_{t+1} = W_t - \frac{\eta}{\sqrt{\hat{V}_t} + \epsilon} \cdot \hat{m}_t$
--

Take the momentum from previous directions along with the direction of latest gradient.

Bias Correction in Adam

$$\begin{aligned} m_t &= \beta \cdot m_{t-1} + (1 - \beta)g_t \quad \rightarrow \nabla W_t \\ \Rightarrow m_0 &= 0 \\ m_1 &= (1 - \beta)g_1 \\ m_2 &= \beta(1 - \beta)g_1 + (1 - \beta)g_2 \\ m_3 &= \beta^2(1 - \beta)g_1 + \beta(1 - \beta)g_2 + (1 - \beta)g_3 \\ \Rightarrow m_t &= (1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i \end{aligned}$$

Expectation:

$$\begin{aligned} E[m_t] &= E \left[(1 - \beta) \sum_{i=1}^t \beta^{t-i} g_i \right] \\ &= E(g_t) \cdot (1 - \beta) \sum_{i=1}^t \beta^{t-i} \Rightarrow \frac{1 - \beta^t}{1 - \beta} \\ &\Rightarrow E[m_t] = (1 - \beta^t)E[g_t] \\ E[m_t]_{\text{expected}} &= E[g_t] \\ E[m_t]_{\text{received}} &= (1 - \beta^t)E[g_t] \\ \Rightarrow \hat{m}_t &= \frac{m_t}{1 - \beta^t} \end{aligned}$$

Ensemble Methods

→ Combine the outputs of different models to reduce generalization error.

– The models can correspond to different classifiers, or different instances of a classifier trained with:

1. different hyperparameters
2. different sets of features
3. different samples of training data

- With replacement: **Bagging**

– In bagging, each instance of the classifier makes errors on different test instances, so the probability of error gets distributed across the test instances rather than being concentrated on one.

– Suppose each model makes an error ε_i on a test example, where ε_i is drawn from a 0 mean multivariate normal distribution:

$$\text{Variance} = E[\varepsilon_i^2] = V, \quad \text{Covariance} = E[\varepsilon_i \varepsilon_j] = C$$

– Error in average prediction:

$$= \frac{1}{K} \sum_i \varepsilon_i$$

– Mean Square Error:

$$\text{MSE} = E \left[\left(\frac{1}{K} \sum_i \varepsilon_i \right)^2 \right]$$

$$\Rightarrow \boxed{\text{Mean Square Error} = \frac{1}{K}V + \frac{K-1}{K}C}$$

Independent Errors:

- $C = 0$
- $\text{MSE} = \frac{V}{K} \leq E[\varepsilon_i^2] = V$
- Bagging works

Dependent Errors:

- $C = V$
- $\text{MSE} = V = E[\varepsilon_i^2] = V$
- Bagging does not work

Gradient and Directional Derivatives

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$,

→ gradient of $f = \nabla f = [f_{x_1}, f_{x_2}, \dots, f_{x_n}]^T$, where:

$$\boxed{f_{x_i} = \frac{\partial f}{\partial x_i}}$$

If u & v are two scalar valued functions:

1. $\nabla(au + bv) = a\nabla u + b\nabla v, \quad a, b \in \mathbb{R}$
2. $\nabla(uv) = u\nabla v + v\nabla u$
3. $\nabla\left(\frac{u}{v}\right) = \frac{v\nabla u - u\nabla v}{v^2}$
4. $\nabla u^n = nu^{n-1}\nabla u$

Directional Derivatives

Given $f : D \subset \mathbb{R}^2 \rightarrow \mathbb{R}$, a unit vector $u = \langle a, b \rangle \in \mathbb{R}^2$:

$$\begin{aligned} D_u f(x_0, y_0) &= \lim_{h \rightarrow 0} \frac{f(x_0 + ah, y_0 + bh) - f(x_0, y_0)}{h} \\ &= \lim_{h \rightarrow 0} \frac{f(x_0 + ah, y_0 + bh) - f(x_0, y_0 + bh) + f(x_0, y_0 + bh) - f(x_0, y_0)}{h} \\ &= \lim_{h \rightarrow 0} \frac{f(x_0 + ah, y_0 + bh) - f(x_0, y_0 + bh)}{ah} \cdot a + \lim_{h \rightarrow 0} \frac{f(x_0, y_0 + bh) - f(x_0, y_0)}{bh} \cdot b \\ &\Rightarrow D_u = f_x(x_0, y_0) \cdot a + f_y(x_0, y_0) \cdot b \\ &\Rightarrow \boxed{D_u f(x_0, y_0) = \nabla f(x_0, y_0) \cdot u = \|\nabla f(x_0, y_0)\| \|u\| \cos \theta} \end{aligned}$$

Gradient Ascent / Descent Cases

- If $\theta = 0^\circ$, the f value will be maximized fastest → Gradient Ascent.
- If $\theta = \pi$, the f value will be minimized fastest → Gradient Descent.
- If $\theta = \pi/2$: Saddle point.

$$\Rightarrow x_{k+1} = x_k - \eta_k \nabla f(x_k)$$

Optimal Learning Rate

Taylor expansion of $J(a)$:

$$J(a) \approx J(a^{(k)}) + \nabla J|_{a^{(k)}}^T (a - a^{(k)}) + \frac{1}{2} (a - a^{(k)})^T H|_{a^{(k)}} (a - a^{(k)})$$

where Hessian:

$$H = \begin{bmatrix} \frac{\partial^2 f}{\partial x_1^2} & \frac{\partial^2 f}{\partial x_1 \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_1 \partial x_n} \\ \frac{\partial^2 f}{\partial x_2 \partial x_1} & \frac{\partial^2 f}{\partial x_2^2} & \cdots & \frac{\partial^2 f}{\partial x_2 \partial x_n} \\ \vdots & \vdots & \ddots & \vdots \\ \frac{\partial^2 f}{\partial x_n \partial x_1} & \frac{\partial^2 f}{\partial x_n \partial x_2} & \cdots & \frac{\partial^2 f}{\partial x_n^2} \end{bmatrix}$$

Let $a = a^{(k+1)} \Rightarrow a^{(k+1)} = a^{(k)} - \eta^{(k)} \nabla J(a^{(k)})$.

$$J(a^{(k+1)}) = J(a^{(k)}) + \nabla J|_{a^{(k)}}^T (a^{(k+1)} - a^{(k)}) + \frac{1}{2} (a^{(k+1)} - a^{(k)})^T H|_{a^{(k)}} (a^{(k+1)} - a^{(k)})$$

Substitute $a^{(k+1)} - a^{(k)} = -\eta^{(k)} \nabla J(a^{(k)})$:

$$J(a^{(k+1)}) = J(a^{(k)}) - \eta^{(k)} \nabla J|_{a^{(k)}}^T \nabla J|_{a^{(k)}} + \frac{1}{2} (\eta^{(k)})^2 \nabla J|_{a^{(k)}}^T H|_{a^{(k)}} \nabla J|_{a^{(k)}}$$

Differentiate w.r.t. $\eta^{(k)}$ and set to zero:

$$\frac{\partial J(a^{(k+1)})}{\partial \eta^{(k)}} = -\nabla J|_{a^{(k)}}^T \nabla J|_{a^{(k)}} + \eta^{(k)} \nabla J|_{a^{(k)}}^T H|_{a^{(k)}} \nabla J|_{a^{(k)}} = 0$$

From the previous step:

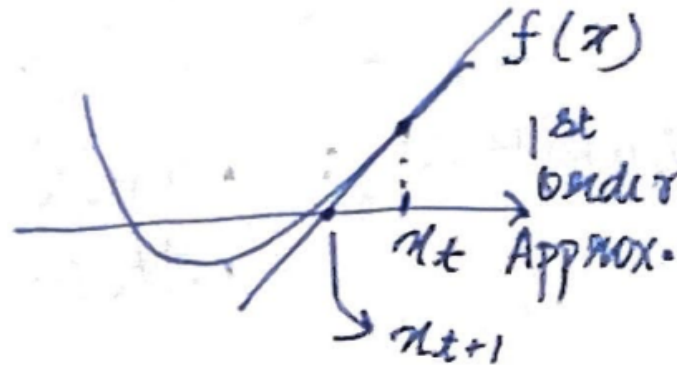
$$\frac{\partial J(a^{(k+1)})}{\partial \eta^{(k)}} = -\nabla J|_{a^{(k)}}^T \nabla J|_{a^{(k)}} + \eta^{(k)} \nabla J|_{a^{(k)}}^T H|_{a^{(k)}} \nabla J|_{a^{(k)}} = 0$$

$$\Rightarrow \eta^{(k)} = \frac{\|\nabla J|_{a^{(k)}}\|^2}{\nabla J|_{a^{(k)}}^T H|_{a^{(k)}} \nabla J|_{a^{(k)}}}$$

Newton's Method for Optimization (2nd Order)

Zero-Finding

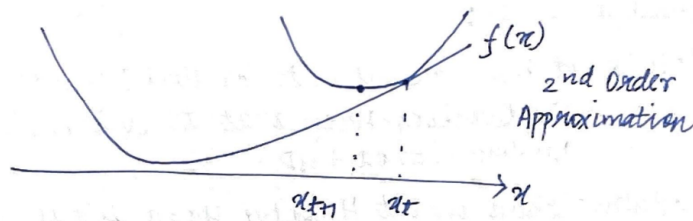
$$x_{t+1} = x_t - \frac{f(x_t)}{f'(x_t)}$$



Optimization

(Zero-finding for $f'(x)$):

$$x_{t+1} = x_t - \frac{f'(x_t)}{f''(x_t)}$$



Idea: Make a 2nd order approximation and optimize that.

Let $f : \mathbb{R}^n \rightarrow \mathbb{R}$ be a sufficiently smooth function. By Taylor's Theorem:

$$f(x) \approx f(a) + g^T(x - a) + \frac{1}{2}(x - a)^T H(x - a) \quad \text{for } x \text{ near } a$$

where:

$$g = \nabla f(a), \quad H = \nabla^2 f(a) = \left(\frac{\partial^2}{\partial x_i \partial x_j} f(a) \right)_{ij}$$

Expand and group similar terms:

$$g(x) = \frac{1}{2}x^T Hx + b^T x + C$$

$$\nabla g(x) = 0 \Rightarrow Hx + b = 0 \Rightarrow \boxed{x = -H^{-1}b = a - H^{-1}g}$$

Caveat: Breaks down if H is not invertible.

From before:

$$\boxed{x = a - H^{-1}g}$$

To check/ensure that x is the minima point, it should be at least PSD:

$$\boxed{\nabla^2 g = H} \Rightarrow \text{minimum if } H \text{ is PSD}$$

Algorithm:

- Initialize $x_0 \in \mathbb{R}^n$
- Iterate: $x_{t+1} = x_t - H^{-1}g$
 - $\nabla f(x_t)$
 - $\nabla^2 f(x_t)$

Possible Issues:

1. H is not PSD $\rightarrow$ Switch to Gradient Descent Use Levenberg–Marquardt to switch between Newton's method & GD.
2. Rather than invert H , solve $Hy = g$ for y , then use:

$$\boxed{x_{t+1} = x_t - y}$$

For numerical & computational efficiency reasons, this is more robust.

3. $x_{t+1} = x_t - \alpha_t y \Rightarrow$ gives more control on the rate of convergence.

Newton's Algorithm

$f : \mathbb{R} \rightarrow \mathbb{R}$

$$x^* = \min_x f(x), \quad x_{k+1} = x_k + t$$

Taylor's Expansion:

$$f(x_k + t) \approx f(x_k) + f'(x_k)t + \frac{1}{2}f''(x_k)t^2$$

$$\frac{d}{dt}f(x_k + t) = 0 \Rightarrow \boxed{t = -\frac{f'(x_k)}{f''(x_k)}}$$

Scalar case:

$$\boxed{x_{k+1} = x_k - \frac{f'(x_k)}{f''(x_k)}}$$

Multi-Dimensional case:

$$x_{k+1} = x_k - H^{-1}(x_k) \nabla f(x_k)$$

Loss Functions (on misclassified samples of w)

Perceptron Criterion:

$$J_p(a) = \sum_{y \in y(a)} -a^T y \rightarrow \geq 0$$

where $y(a)$ is the set containing misclassified samples.

* $J_p(a) = 0$ when there are no misclassified samples.

$$\begin{aligned} \nabla J(a_k) &= - \sum_{y \in y(a)} y \\ \Rightarrow a_{k+1} &= a_k + \eta^{(k)} \sum_{y \in y(a)} y \end{aligned}$$

Example: $J_{\text{sample}}(a) = \# \text{misclassified samples} = |y(a)| \rightarrow$ gradient might not exist at many points (piece-wise constant fn).

Sequential Solvers

$$y^1, y^2, y^3, \dots, y^n$$

$$a_{k+1} = a_k + \eta^{(k)} \sum y \rightarrow \text{GD}$$

$$a_{k+1} = a_k + \eta^{(k)} y_k \rightarrow \text{SGP (single sample perceptron) if misclassified}$$

$\rightarrow$ SGD takes more time to converge but ensures global convergence & is more computationally efficient.

Convergence of SGD

Let $\eta^{(k)} \rightarrow \text{fixed} = 1$, $a(0) = \text{randomly initialized vector}$.

$$a_{k+1} = a_k + y^k$$

$$\|a_{k+1} - \alpha \hat{a}\|^2 \leq \|a_k - \alpha \hat{a}\|^2$$

Criteria for convergence $\rightarrow$ solution: $\hat{a}^T y_i > 0 \forall i \Rightarrow$ no misclassification.

Stopping Criteria:

$$\|a_{k+1} - \alpha \hat{a}\| = 0$$

Update Rule:

$$a_{k+1} = a_k + y^k$$

$$a_{k+1} - \alpha \hat{a} = a_k - \alpha \hat{a} + y^k$$

$$\Rightarrow \|a_{k+1} - \alpha \hat{a}\|^2 = \|a_k - \alpha \hat{a}\|^2 + 2(a_k - \alpha \hat{a})^T y^k + \|y^k\|^2$$

$$\Rightarrow \|a_{k+1} - \alpha \hat{a}\|^2 \leq \|a_k - \alpha \hat{a}\|^2 - 2\alpha \hat{a}^T y^k + \|y^k\|^2$$

For convergence:

$$-2\alpha \hat{a}^T y^k + \|y^k\|^2 \leq 0$$

$$\Rightarrow \alpha \geq \frac{\|y^k\|^2}{2\hat{a}^T y_k}$$

Let $\beta^2 = \max_{\forall i} \|y_i\|^2$, $\gamma = \min_{\forall i} \hat{a}^T y_i > 0$.

$$\Rightarrow \alpha = \frac{\beta^2}{2\gamma} + \delta, \quad \delta > 0$$

$$\|a_{k+1} - \alpha \hat{a}\|^2 \leq \|a_k - \alpha \hat{a}\|^2 - 2 \left(\frac{\beta^2}{2\gamma} + \delta \right) \gamma + \beta^2$$

$$\leq \|a_k - \alpha \hat{a}\|^2 - \delta \gamma$$

We are going closer to $\hat{a}$ in this iteration by at least this amount.

$$\|a_{k+1} - \alpha \hat{a}\|^2 \leq \|a(0) - \alpha \hat{a}\|^2 - (k+1)\xi$$

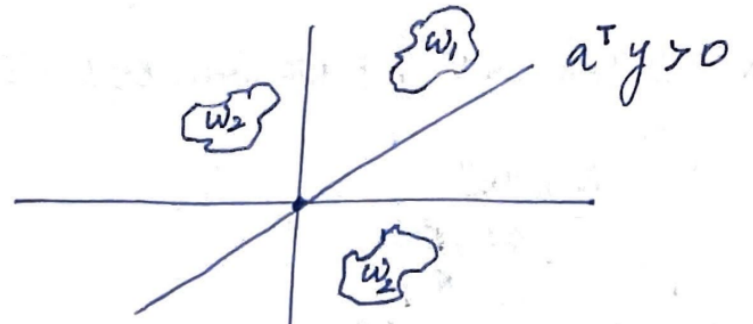
At the stopping point:

$$\|a(0) - \alpha \hat{a}\|^2 - (K+1)\xi = 0$$

$$\Rightarrow K = \frac{\|a(0) - \alpha \hat{a}\|^2}{\xi} - 1$$

Note: This is a theoretical bound since $\hat{a}$ is not even known. If $\hat{a}$ had been known, we wouldn't be doing any of this.

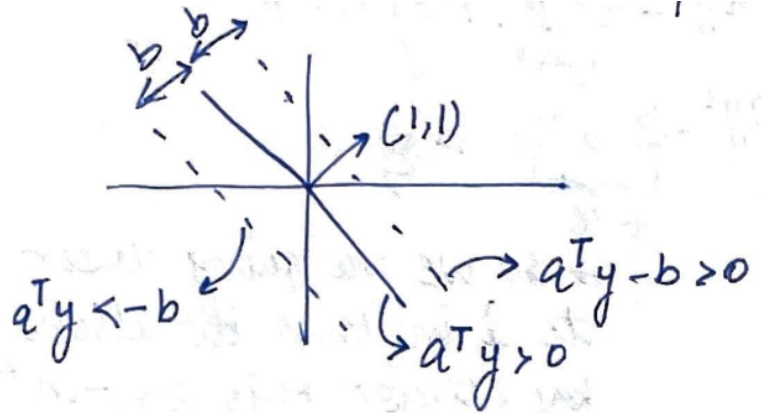
Relaxation Criteria



Can we do better?

$$a^T y > b, \quad b > 0$$

(Restricting the solution region) where y on reflected samples of w_2 , x on samples of w_1 .



$a^T y > 0$ represents no minimum distance from boundary.

$a^T y > b$ means the closest element to the decision boundary will be at least at distance b .

Effect on actual samples of w_2 ?

$$a^T(-y) > b \quad \Rightarrow \quad a^T(y) < -b$$

Difference between properly & safely classified.

$$J_r(a) = \frac{1}{2} \sum_{y \in y(a)} \frac{(a^T y - b)^2}{\|y\|^2}$$

where $y(a)$ = set of misclassified samples for a .

$$\nabla J_r(a) = \sum_{y \in y(a)} \frac{a^T y - b}{\|y\|^2} \cdot y$$

G.D. Rule

$$a_{k+1} = a_k + \eta(k) \sum_{y \in y(a)} \frac{b - a^T y}{\|y\|^2} y$$

Single Sample Correction Rule

$$a_{k+1} = a_k + \eta(k) \frac{b - a^T y}{\|y\|^2} y$$

Mean Squared Error

$$y = \begin{bmatrix} 1 \\ x \end{bmatrix}, \quad a^T y_i = b_i \quad (> 0)$$

$$\begin{bmatrix} y_{10} & y_{11} & \cdots & y_{1d} \\ y_{20} & y_{21} & \cdots & y_{2d} \\ \vdots & & & \vdots \\ y_{n0} & y_{n1} & \cdots & y_{nd} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{bmatrix} = \begin{bmatrix} b_1 \\ b_2 \\ \vdots \\ b_n \end{bmatrix}$$

$$Ya = b$$

$$\begin{aligned} &\rightarrow \boxed{e = Ya - b} \quad (\text{error}) \\ &\rightarrow J_s(a) = \|Ya - b\|^2, \quad \nabla J_s(a) = 0 \\ &* \quad S = S(x), \quad r = r(x) \\ &\rightarrow \frac{\partial}{\partial x} S^T a^T r = \left[\frac{\partial S}{\partial x} \right]^T a^T r + \left[\frac{\partial r}{\partial x} \right]^T a^T S \\ &\Rightarrow \boxed{\nabla_a J_s(a) = 2Y^T(Ya - b)} \end{aligned}$$

Analytical Form

$$2Y^T(Ya - b) = 0 \quad \Rightarrow \quad Y^T Ya = Y^T b$$

$$\Rightarrow \{a = (Y^T Y)^{-1} Y^T b = Y^\dagger b \quad (\text{pseudo inverse})\}$$

$$* \quad Y^\dagger Y = I, \quad YY^\dagger \neq I \text{ If } Y^T Y \text{ is not invertible, we go for iterative solvers}$$

Gradient Solver for MSE

$$a_{k+1} = a_k - \eta(k) Y^T (Ya(k) - b)$$

Widrow–Hoff Procedure

Single sample correction for MSE criterion:

$$a_{k+1} = a_k + \eta(k) (b_k - a_k^T y_k) y_k$$

$$a_k^T y_k \neq b_k$$

$$\begin{aligned} * \quad a_k^T y_k < b_k &\Rightarrow a_{k+1} \text{ moves toward } y_k \\ a_k^T y_k > b_k &\Rightarrow a_{k+1} \text{ moves away from } y_k \end{aligned}$$

rightarrow Here, we are assuming that b is already known. In reality however, we will have to find b as well $\rightarrow$ *The classification margin.*

Ho–Kashyap Algorithm

$$J_{HK}(a, b) = \|Ya - b\|^2 \quad b > 0$$

- $\nabla_a J_{HK}(a, b) = 2Y^T(Ya - b)$
- $\nabla_b J_{HK}(a, b) = -2(Ya - b)$

Update Rule for a :

$$a_{k+1} = a_k - \eta(k) Y^T (Y a_k - b)$$

Update Rule for b :

$$b_{k+1} = b_k - \eta(k) \frac{1}{2} [\nabla_b J_{HK}(a(k), b(k)) - |\nabla_b J_{HK}(a(k), b(k))|]$$

modified G.D. $\rightarrow$ no update when $J_{HK} > 0 \Rightarrow$ ensures b remains > 0 .

$\rightarrow x_1, x_2, \dots, x_n : n$ # of d -dim features

$$y_i = \begin{bmatrix} 1 \\ x_i \end{bmatrix} = \begin{bmatrix} y_{i0} \\ y_{i1} \\ \vdots \\ y_{id} \end{bmatrix}, \quad a = \begin{bmatrix} w_0 \\ w \end{bmatrix} = \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_d \end{bmatrix}$$

• Assume n_1 # of samples $\rightarrow D_1$ with label w_1 , $n_2 = (n - n_1)$ # of samples $\rightarrow D_2$ with label w_2 .

$$b = \begin{bmatrix} \frac{n}{n_1} \mathbf{1}_1 \\ \frac{n}{n_2} \mathbf{1}_2 \end{bmatrix}$$

Solve for a using MSE criterion

$\rightarrow Y^T Y a = Y^T b$

$$\begin{aligned} Y &= \begin{bmatrix} \mathbf{1}_{n_1 \times 1} & X_{1_{n_1 \times d}} \\ -\mathbf{1}_{n_2 \times 1} & -X_{2_{n_2 \times d}} \end{bmatrix} \\ Y^T Y &= \begin{bmatrix} \mathbf{1}_{n_1 \times 1}^T & -\mathbf{1}_{n_2 \times 1}^T \\ X_{1_{n_1 \times d}}^T & -X_{2_{n_2 \times d}}^T \end{bmatrix} \begin{bmatrix} \mathbf{1}_{n_1 \times 1} & X_{1_{n_1 \times d}} \\ -\mathbf{1}_{n_2 \times 1} & -X_{2_{n_2 \times d}} \end{bmatrix} \\ &\Rightarrow \begin{bmatrix} \mathbf{1}_1^T & -\mathbf{1}_2^T \\ X_1^T & -X_2^T \end{bmatrix} \begin{bmatrix} \mathbf{1}_1 & X_1 \\ -\mathbf{1}_2 & -X_2 \end{bmatrix} \begin{bmatrix} W_0 \\ W \end{bmatrix} = \begin{bmatrix} \mathbf{1}_1^T & -\mathbf{1}_2^T \\ X_1^T & -X_2^T \end{bmatrix} \begin{bmatrix} \frac{n}{n_1} \mathbf{1}_1 \\ \frac{n}{n_2} \mathbf{1}_2 \end{bmatrix} \\ &\quad \begin{bmatrix} 0 \\ \frac{n}{n_1} X_1^T \mathbf{1}_1 - \frac{n}{n_2} X_2^T \mathbf{1}_2 \end{bmatrix} = \begin{bmatrix} \frac{n}{n_1} \mathbf{1}_1^T \mathbf{1}_1 - \frac{n}{n_2} \mathbf{1}_1^T \mathbf{1}_2 \\ \frac{n}{n_1} X_1^T \mathbf{1}_1 - \frac{n}{n_2} X_2^T \mathbf{1}_2 \end{bmatrix} \\ X_1^T \mathbf{1}_1 &= \begin{bmatrix} - & - & - & - \\ - & - & - & - \\ & & \vdots & \\ - & - & - & - \end{bmatrix}^T \begin{bmatrix} \vdots \end{bmatrix}_{1_1} = \begin{bmatrix} \left| \right| & \left| \right| & \left| \right| & \left| \right| & \dots & \left| \right| \end{bmatrix} \begin{bmatrix} \vdots \end{bmatrix}_{1_1} = \sum_{X \in D_1} X \end{aligned}$$

(each line is a feature vector)

$$\begin{aligned} &\Rightarrow \begin{bmatrix} \mathbf{1}_1^T & -\mathbf{1}_2^T \\ X_1^T & -X_2^T \end{bmatrix} \begin{bmatrix} \mathbf{1}_1 & X_1 \\ -\mathbf{1}_2 & -X_2 \end{bmatrix} \begin{bmatrix} W_0 \\ W \end{bmatrix} = \begin{bmatrix} 0 \\ \frac{n}{n_1} \sum_{X \in D_1} X - \frac{n}{n_2} \sum_{X \in D_2} X \end{bmatrix} \\ &\quad \begin{bmatrix} n_1 + n_2 & \sum_{X \in D_1} X^T + \sum_{X \in D_2} X^T \\ X_1^T X_1 + X_2^T X_2 & \end{bmatrix} \begin{bmatrix} W_0 \\ W \end{bmatrix} = \begin{bmatrix} 0 \\ nm_1 - nm_2 \end{bmatrix} \end{aligned}$$

Let

$$\begin{aligned} A &= \sum_{X \in D_i} X X^T = \sum_{X \in D_i} \begin{bmatrix} x_{i1} \\ x_{i2} \\ \vdots \\ x_{id} \end{bmatrix} [x_{i1} \quad x_{i2} \quad \dots \quad x_{id}] \\ &\Rightarrow A = \sum_{X \in D_i} \begin{bmatrix} x_{i1}^2 & x_{i1}x_{i2} & \dots & x_{i1}x_{id} \\ x_{i2}x_{i1} & x_{i2}^2 & \dots & x_{i2}x_{id} \\ \vdots & \vdots & \ddots & \vdots \\ x_{id}x_{i1} & x_{id}x_{i2} & \dots & x_{id}^2 \end{bmatrix} \\ &= \begin{bmatrix} A_{11} & A_{12} & \dots & A_{1d} \\ \vdots & \vdots & \ddots & \vdots \\ A_{d1} & A_{d2} & \dots & A_{dd} \end{bmatrix} \end{aligned}$$

$$\begin{aligned} A_{11} &= x_{11}^2 + x_{21}^2 + \dots + x_{n_1,1}^2 \\ A_{12} &= x_{11}x_{12} + x_{21}x_{22} + \dots + x_{n_1,1}x_{n_1,2} \end{aligned}$$

$$\Rightarrow X_1^T X_1 = \begin{bmatrix} x_{11} & \cdots & x_{n_1 1} \\ x_{12} & \cdots & x_{n_1 2} \\ \vdots & \ddots & \vdots \\ x_{1d} & \cdots & x_{n_1 d} \end{bmatrix} \begin{bmatrix} x_{11} & x_{12} & \cdots & x_{1d} \\ \vdots & \vdots & \ddots & \vdots \\ x_{n_1 1} & x_{n_1 2} & \cdots & x_{n_1 d} \end{bmatrix}$$

(n_1 d -dimensional points belonging to class ω_1)

$$\Rightarrow \boxed{X_1^T X_1 + X_2^T X_2 = \sum_{X \in D_i} X X^T} \quad (\text{looks similar to the total within-class scatter})$$

$$\begin{aligned} &\rightarrow \boxed{S_W = \sum_{i=1}^2 \sum_{X \in D_i} (X - m_i)(X - m_i)^T} \rightarrow \text{Total within-class scatter} \\ &= \sum_{i=1}^2 [\sum_{X \in D_i} (X X^T - X m_i^T - m_i X^T + m_i m_i^T)] = X_1 X_1^T + X_2 X_2^T + \sum_{i=1}^2 (-n_i m_i m_i^T - n_i m_i m_i^T + n_i m_i m_i^T + n_i m_i m_i^T) \\ &\Rightarrow \boxed{X_1 X_1^T + X_2 X_2^T = S_W + n_1 m_1 m_1^T + n_2 m_2 m_2^T} \end{aligned}$$

$$\begin{aligned} &\Rightarrow \begin{bmatrix} n & (n_1 m_1 + n_2 m_2)^T \\ n_1 m_1 + n_2 m_2 & S_W + n_1 m_1 m_1^T + n_2 m_2 m_2^T \end{bmatrix} \begin{bmatrix} W_0 \\ W \end{bmatrix} = \begin{bmatrix} 0 \\ n(m_1 - m_2) \end{bmatrix} \\ &\rightarrow n W_0 + (n_1 m_1 + n_2 m_2)^T W = 0 \end{aligned}$$

$$\Rightarrow \boxed{W_0 = -\frac{(n_1 m_1 + n_2 m_2)^T W}{n} = -m^T W} \quad (1)$$

$$\rightarrow (n_1 m_1 + n_2 m_2) \left[-\frac{(n_1 m_1 + n_2 m_2)^T W}{n} \right] + S_W W + n_1 m_1 m_1^T W + n_2 m_2 m_2^T W = n(m_1 - m_2)$$

$$\Rightarrow \boxed{\left[S_W + (n_1 m_1 + n_2 m_2) \left(-\frac{(n_1 m_1 + n_2 m_2)^T}{n} \right) + n_1 m_1 m_1^T + n_2 m_2 m_2^T \right] W = n(m_1 - m_2)} \quad (2)$$

$$\begin{aligned} &\Rightarrow \left[S_W - \frac{n_1^2 m_1 m_1^T + n_1 n_2 m_1 m_2^T + n_1 n_2 m_2 m_1^T + n_2^2 m_2 m_2^T}{n} + n_1 m_1 m_1^T + n_2 m_2 m_2^T \right] W = n(m_1 - m_2) \\ &\Rightarrow \left[S_W + \frac{n_1 n_2}{n} [m_1 m_1^T + m_2 m_2^T - m_1 m_2^T - m_2 m_1^T] \right] W = n(m_1 - m_2) \\ &\Rightarrow \left[\frac{1}{n} S_W + \frac{n_1 n_2}{n^2} (m_1 - m_2)(m_1 - m_2)^T \right] W = (m_1 - m_2) \\ &\Rightarrow \frac{1}{n} S_W W + \alpha (m_1 - m_2) = (m_1 - m_2) \\ &\Rightarrow S_W W = \alpha' (m_1 - m_2) \\ &\Rightarrow \boxed{W = \alpha S_W^{-1} (m_1 - m_2)} \end{aligned}$$

LDA is a special case of MSE

Maximum Margin Linear SVM

$$g(x) = a^T x + b, \quad x_1, x_2, \dots, x_n \in \mathbb{R}^d$$

Discriminant Function:

$$\begin{aligned} &\begin{cases} g(x) = a^T x + b > 0 & \Rightarrow \text{class } \omega_1, \text{ label } t_n = 1 \\ g(x) = a^T x + b < 0 & \Rightarrow \text{class } \omega_2, \text{ label } t_n = -1 \end{cases} \\ &\Rightarrow \boxed{t_n g(x) > 0} \end{aligned}$$

Margin: Minimum distance of any sample to the decision boundary.

Hard Margin

There exists a margin > 0 strictly.

1. Find out distances $\forall x_i$
2. Find minimum of those distances $\rightarrow$ Margin
3. Maximize the minimum distance

Distance of any point x_n from H :

$$\text{dist} = \frac{t_n g(x_n)}{\|W\|} = \frac{t_n(W^T x_n + b)}{\|W\|} \quad (\|W\| \neq 0)$$
$$\max_{W,b} \left[\min_n \frac{1}{\|W\|} t_n(W^T x_n + b) \right]$$

$\rightarrow$ The point(s) with minimum distance are called **Support Vectors**.

Primal Form (Difficult to solve):

$$\max_{W,b} \frac{1}{\|W\|} \quad \text{s.t.} \quad t_n(W^T x_n + b) \geq 1, \quad \forall n$$

Alternative Formulation

Scale: $W \rightarrow kW$, $b \rightarrow kb$, $k > 0$ d remains the same, but $t_n(W^T x_n + b)$ changes. This gives a constraint:

$$t_n(W^T x_n + b) \geq 1 \quad (\text{equality for support vectors})$$

$$\boxed{\arg \max_{W,b} \frac{1}{\|W\|} \quad \text{s.t.} \quad t_n(W^T x_n + b) \geq 1, \quad \forall n}$$

$$\boxed{\arg \min_{W,b} \frac{\|W\|^2}{2} \quad \text{s.t.} \quad t_n(W^T x_n + b) \geq 1, \quad \forall n}$$

$\rightarrow$ **Inequality Constraint:** KKT conditions apply.

Norm² problems are convex problems:

$$L(W, b, \alpha) = \frac{1}{2} \|W\|^2 - \sum_{n=1}^N \alpha_n [t_n(W^T x_n + b) - 1], \quad \alpha_n \geq 0$$
$$\min_{W,b} \max_{\alpha_n \geq 0} L(W, b, \alpha)$$

Non-linearly Separable Case

If $t_n(W^T x_n + b) \leq 0$: α_n takes a large +ve value to penalize L .

$$\Rightarrow \alpha_n \rightarrow \infty \quad (\|W\| \text{ fixed})$$

α_n tries to push t_n because it is a penalizing term, while W, b try to minimize it.

Hard Margin: If classes are not separable, $L \rightarrow \infty$ and the problem cannot be solved.

$$\boxed{\min_{\alpha_n} L(W, b, \alpha) = \frac{1}{2} \|W\|^2 - \sum_{n=1}^N \alpha_n \delta_n} \rightarrow (\text{Primal Form})$$

Solve and cross-check α_n, W, b because Lagrange gives solution for equality condition: $t_n(W^T x + b) = 1$

Inequality Constraints for Convex Problems

$$f_i : \mathbb{R}^n \rightarrow \mathbb{R} \quad (\text{scalar-valued function, argument can be vector})$$

$\rightarrow$ Each f_i is convex.

Problem: $\min f_0(x)$ subject to $f_i(x) \leq 0, \quad 1 \leq i \leq m$

Loss Function:

$$\mathcal{L}(x, \lambda) = \sum_{i=0}^m \lambda_i f_i(x), \quad \lambda = (\lambda_0, \lambda_1, \dots, \lambda_m)$$

KKT Conditions (Slater Point)

For $\lambda_0 = 1$, the following conditions hold at the point of optimality x^* :

1. $\nabla_x \mathcal{L}(x^*, \lambda) = 0^n$
2. $\lambda_i \geq 0, \quad 1 \leq i \leq m$
3. $\lambda_i f_i(x^*) = 0, \quad 1 \leq i \leq m$

$$\min_{W, b} \frac{1}{2} \|W\|^2 \quad \text{s.t.} \quad t_n(W^T x_n + b) \geq 1$$

$$\Rightarrow \min_{W, b} L(W, b, \alpha) = \frac{1}{2} \|W\|^2 - \sum_{n=1}^N \alpha_n [t_n(W^T x_n + b) - 1]$$

$$1) \frac{\partial L}{\partial W} = 0, \quad 2) \frac{\partial L}{\partial b} = 0$$

$$\Rightarrow \boxed{W = \sum_{n=1}^N \alpha_n t_n x_n}$$

$$\sum_{n=1}^N \alpha_n t_n = 0$$

$$\Rightarrow L(\alpha) = \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m t_n t_m x_n^T x_m$$

$$x_n^T x_m \rightarrow \phi(x_n)^T \phi(x_m) \Rightarrow \text{Kernel SVM: } a^T \phi(x) + b$$

Soft Margin Linear SVM

$$L(W, b, \alpha) = \frac{\|W\|^2}{2} - \sum_{n=1}^N \alpha_n [t_n(W^T x_n + b) - 1]$$

becomes $+ve$ since $\alpha \geq 0$

$$\Rightarrow L \rightarrow \infty$$

→ Need to introduce a slack variable ξ_n to fix this issue as we might not always have linearly separable classes.

→ For correctly classified samples: $t_n(W^T x_n + b) \geq 1 - \xi_n \Rightarrow \xi_n = 0$

→ For wrongly classified samples: $t_n(W^T x_n + b) \geq 1 - \xi_n \Rightarrow \xi_n > 0$

$$\boxed{\xi_n \geq 0} \quad \text{Conveniently shifts the boundary for wrongly classified}$$

* But we want ξ_n to be equal to 0 in most cases. This means we want to get the best possible classification line assuming that it is a hard margin SVM. → We want to use the slack variable only when absolutely necessary.

$$\boxed{\min_{W, b, \xi} \frac{\|W\|^2}{2} + C \sum_{n=1}^N \xi_n \quad \text{s.t.} \quad \xi_n \geq 0, \quad t_n(W^T x_n + b) \geq 1 - \xi_n}$$

$$\boxed{L(W, b, \xi, \alpha, \mu) = \frac{\|W\|^2}{2} + C \sum_{n=1}^N \xi_n - \sum_{n=1}^N \alpha_n \{t_n(W^T x_n + b) - 1 + \xi_n\} - \sum_{n=1}^N \mu_n \xi_n}$$

KKT Conditions:

1. $\alpha_n \geq 0$
2. $t_n(W^T x_n + b) - 1 + \xi_n \leq 0$
3. $\alpha_n (t_n(W^T x_n + b) - 1 + \xi_n) = 0$
4. $\mu_n \geq 0$
5. $\xi_n \geq 0$

6. $\mu_n \xi_n = 0$

- $\frac{\partial L}{\partial W} = 0 \Rightarrow \boxed{W = \sum_{n=1}^N \alpha_n t_n x_n}$

- $\frac{\partial L}{\partial b} = 0 \Rightarrow \boxed{\sum_{n=1}^N \alpha_n t_n = 0}$

- $\frac{\partial L}{\partial \xi_n} = 0 \Rightarrow C - \alpha_n - \mu_n = 0 \Rightarrow \boxed{\alpha_n = C - \mu_n}$

$$\Rightarrow L(\alpha) = \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m t_n t_m x_n^T x_m + \sum_{n=1}^N (C - \mu_n) \xi_n - \sum_{n=1}^N \alpha_n (t_n (W^T x_n + b) - 1 + \xi_n)$$

$$\Rightarrow L(\alpha) = \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m t_n t_m x_n^T x_m + \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m t_n t_m x_n^T x_m$$

$$\boxed{L(\alpha) = \sum_{n=1}^N \alpha_n - \frac{1}{2} \sum_{n=1}^N \sum_{m=1}^N \alpha_n \alpha_m t_n t_m x_n^T x_m}$$

Same as hard margin? $\rightarrow$ The optimal equation comes out to be identical to the hard margin equation, but the conditions used to derive them are different, and the soft margin's α_n satisfies more constraints.

Soft Margin Kernel SVM: $a^T \phi(x) + c$

$$x_n^T x_m \rightarrow \phi(x_n)^T \phi(x_m)$$

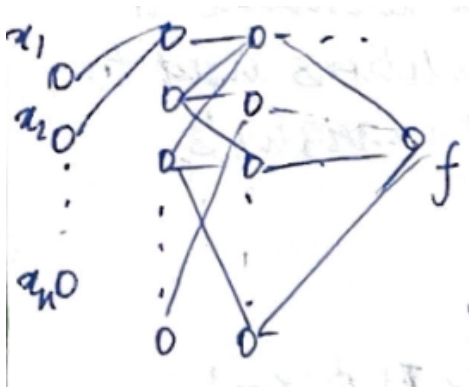
Neural Networks

Arnold Kolmogorov Theorem

Any multivariate continuous fn f can be represented as a superposition of *2-argument addition* of continuous fns of one variable.

$$f(x_1, x_2, \dots, x_n) = \sum_{j=0}^{2n} \phi_j \left(\sum_{i=1}^n \psi_{i,j}(x_i) \right), \text{ where } \phi_j: \text{ dependent on } f \quad \psi_{i,j}: \text{ independent of } f$$

Arnold Kolmogorov Theorem: You don't need to approximate the # of nodes $S(n)$, if you know ϕ_j & $\psi_{i,j}$, because if these



are known, the problem of # of nodes becomes finite.

$\rightarrow$ But the issue is that we have no information about the nature of ϕ & ψ .

Universal Approximation Theorem

$$f(x_1, x_2, \dots, x_n) \simeq \sum_{j=1}^N \alpha_j \sigma(W_j^T x + b_j)$$

non-linear activation fn

→ A neural network with just one hidden layer is enough to approximate a continuous fn for inputs within a specific range.
 ↪ Possible due to the use of **Non-linear** activation fns such as sigmoid, tanh, ReLU, etc.

C-class Classification Problem

$$X \in \mathbb{R}^d \\ g(x_i) > g(x_j) \quad \forall j \neq i : \text{class } \omega_i$$

Terminology:

i^{th} layer → i/p layer node i
 j^{th} layer → hidden layer node j
 k^{th} layer → o/p layer node k

$$\text{net}_j = \sum_{i=1}^d x_i w_{ji} + w_{j0} = w_j^T X$$

$$y_j = f(\text{net}_j) \quad (\text{activation fn}) \quad \left| \quad \text{net}_k = \sum_{j=0}^{n_H} y_j w_{kj} + w_{k0} \right.$$

output $z_k = f(\text{net}_k)$

$$\Rightarrow g_k(X) = f\left(\sum_{j=0}^{m_H} w_{kj} f\left(\sum_{i=1}^d w_{ji} x_i + w_{j0}\right) + w_{k0}\right)$$

Prediction: $\arg \max_j g_k(X_i), j \in \{1, 2, \dots, C\}$

XOR Problem

$$f = \text{sgn}(x) = \begin{cases} 1, & \geq 0 \\ -1, & < 0 \end{cases}$$

$$y = \begin{cases} 1, & -0.4x_1 + 0.7x_2 \geq -1 \\ -1, & \text{o.w.} \end{cases} \quad f(x_1 + x_2 - 1.5) = \begin{cases} 1, & x_1 + x_2 \geq 1.5 \\ -1, & \text{otherwise} \end{cases}$$

Cases:

$$\begin{aligned} x_1 = x_2 = 1 &\rightarrow y = -1 \\ x_1 = x_2 = -1 &\rightarrow y = -1 \\ x_1 = 1, x_2 = -1 &\rightarrow y = 1 \\ x_1 = -1, x_2 = 1 &\rightarrow y = 1 \end{aligned}$$

Backpropagation

$X \rightarrow t$ one-hot vectors

$$\begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad \text{cat} \quad \text{dog} \quad \text{tiger}$$

$$J(W) = \frac{1}{2} \sum_{k=1}^C (t_k - z_k)^2 \quad \Rightarrow \quad W_{k+1} = W_k - \eta \nabla_W J$$

For W_{kj} :

$$\frac{\partial J}{\partial W_{kj}} = \frac{\partial J}{\partial z_k} \cdot \frac{\partial z_k}{\partial \text{net}_k} \cdot \frac{\partial \text{net}_k}{\partial W_{kj}} = -(t_k - z_k) f'(\text{net}_k) y_j$$

$$\Delta W_{kj} = -\eta \frac{\partial J}{\partial W_{kj}} = \eta (t_k - z_k) f'(\text{net}_k) y_j \quad \delta_k \rightarrow \text{sensitivity}$$

For W_{ji} :

$$\frac{\partial J}{\partial W_{ji}} = \frac{\partial J}{\partial y_j} \cdot \frac{\partial y_j}{\partial \text{net}_j} \cdot \frac{\partial \text{net}_j}{\partial W_{ji}} = \frac{\partial J}{\partial y_j} f'(\text{net}_j) x_i$$

$$\frac{\partial J}{\partial y_j} = \sum_{k=1}^C \frac{\partial J}{\partial z_k} \cdot \frac{\partial z_k}{\partial \text{net}_k} \cdot \frac{\partial \text{net}_k}{\partial y_j} = - \sum_{k=1}^C (t_k - z_k) f'(\text{net}_k) W_{kj}$$

$$\Delta W_{ji} = -\eta \frac{\partial J}{\partial W_{ji}} = \left\{ \sum_{k=1}^C (t_k - z_k) f'(\text{net}_k) W_{kj} \right\} f'(\text{net}_j) x_i$$

Training / Validation / Test

dataset $\rightarrow$ *training* | *val* | *test*
 learn the weights model selection

Bias & Variance

High Bias

- Underfit
- Low train & test accuracies

High Variance

- Overfit
- High train but low test accuracies

Corrective Measures

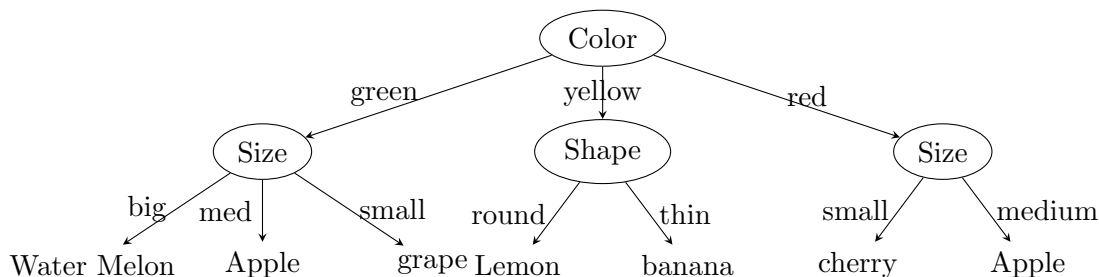
High Bias (Underfit)

1. Add more features
2. Increase model complexity (Polynomial features)
3. Decrease Regularization

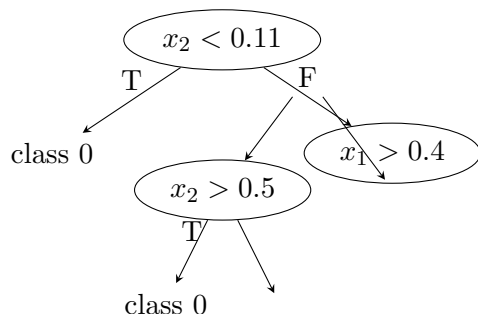
High Variance (Overfit)

1. Reduce features
2. Add more training data
3. Increase Regularization
4. Decrease model complexity
5. Add Dropouts

(Directed) Decision Trees



1. Direction of Decision Making: Parent node $\rightarrow$ Child node
2. Not unique typically; needs to be constructed
3. Criteria:

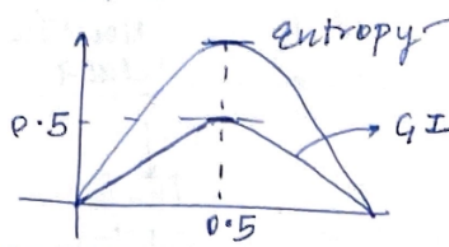


This algorithm is called **Recursive Binary Partitioning**.

$\rightarrow$ Decision trees can be extended as-is to multiclass classification problems without any change.

Gini Impurity in DTs

$$\text{GI} = \sum_{i=1}^C p_i(1 - p_i) \rightarrow \text{probability of class } i \text{ being selected in the partition.}$$



Entropy requires log calculations (slower), GI is more efficient computationally.

Weighted Gini Impurity:

$$\text{Weighted GI} = \frac{n_L G_1}{M} + \frac{n_R G_2}{M}$$

n_L, n_R : points in left/right partition

M : Total number of points

Default if criterion not given.

Information Gain

$$\text{Entropy } E = - \sum_{i=1}^C p_i \log_2 p_i \rightarrow p_i: \text{Probability of randomly picking samples of class } i$$

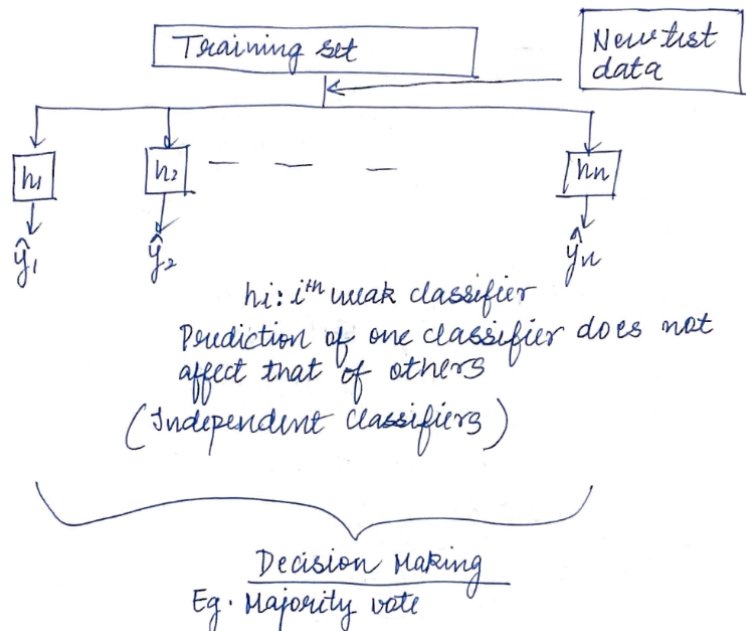
$$\text{Information Gain} = E(T) - E(T|a)$$

Entropy of parent partition minus total entropy for cut a (weighted sum).

Ensembling of Classifiers

– Use multiple weak classifiers to make a decision better than each classifier individually.

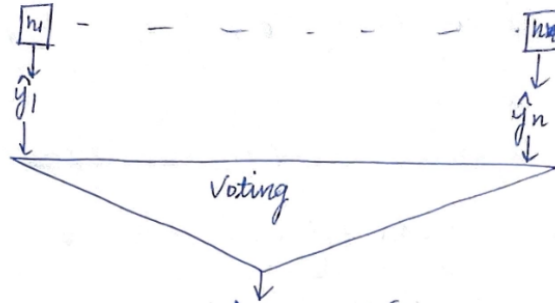
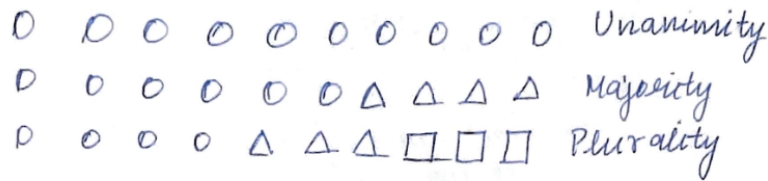
→ Integration Strategy



Error rate of h_i :

$$E_i = \frac{\# \text{ misclassifications}}{\# \text{ samples}}$$

Majority Voting Classifier



$$\hat{y}_f = \text{mode}\{h_1(x), h_2(x), \dots, h_n(x)\}$$

Assume n independent classifiers with base error rate $\epsilon < 0.5$ (better than random guess).

Probability of correct prediction by ensemble:

$$P(CK) = \binom{n}{K} \epsilon^K (1 - \epsilon)^{n-K}$$

Ensemble error:

$$E = \sum_{K=\lceil n/2 \rceil}^n \binom{n}{K} \epsilon^K (1 - \epsilon)^{n-K}$$

Soft Voting

$$\hat{y} = \arg \max_j \sum_{i=1}^n w_i p_{i,j}$$

Optional weighting parameter

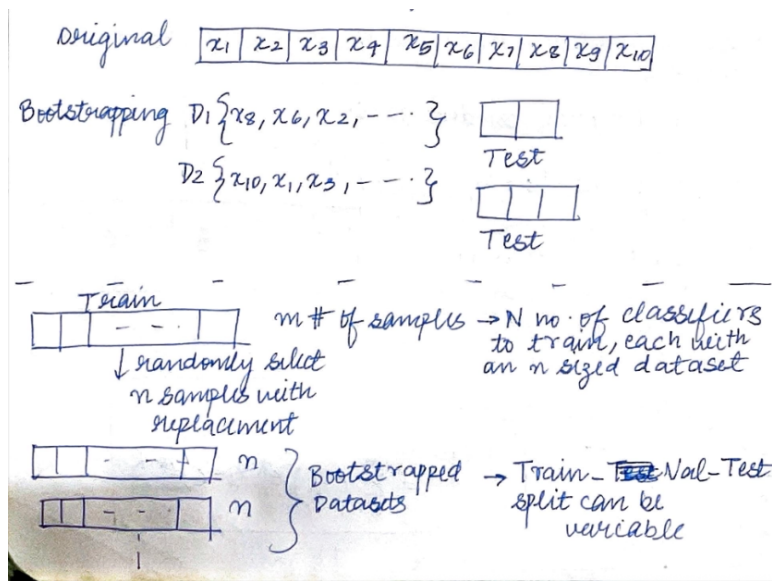
– higher for more confident classifiers.

– default: $w_i = \frac{1}{n} \forall w_i \in \{w_1, \dots, w_n\}$

Bootstrapping

For training independent classifiers on a dataset.

m # of samples in dataset $\rightarrow$ Create a dataset with n samples by picking each sample randomly with replacement.



Boosting

- Selection prob changes at each round.
- Change the prob of selection for samples.
- Change the confidence of the classifier.

Adaptive Boosting (AdaBoost) Algorithm

- **Initialize K:** # of Adaboost rounds (# of weak classifiers)
- **Initialize D:** Train set $D = \{(x^{(i)}, y^{(i)})\}$, $y_i \in \{-1, 1\}$.
- **Training:-**

for $r = 1$ to K :

$$\forall i : W_r(i) = \frac{W_r(i)}{\sum_{\forall i} W_r(i)}$$

*pmf at every step

h_r : fit weak classifier (D, W_n) $\rightarrow$ Bootstrapping

$$\epsilon_r = \sum_{\forall i} W_r(i) \mathbb{1}(h_r(x_i) \neq y_i)$$

if $\epsilon_r > \frac{1}{2}$: ignore

else:

$$\alpha_r = \frac{1}{2} \log \left[\frac{(1 - \epsilon_r)}{\epsilon_r} \right] \rightarrow \text{Confidence term}$$

$$W_{r+1}(i) = W_r(i) \cdot \exp[-\alpha_r y_i h_r(x_i)]$$

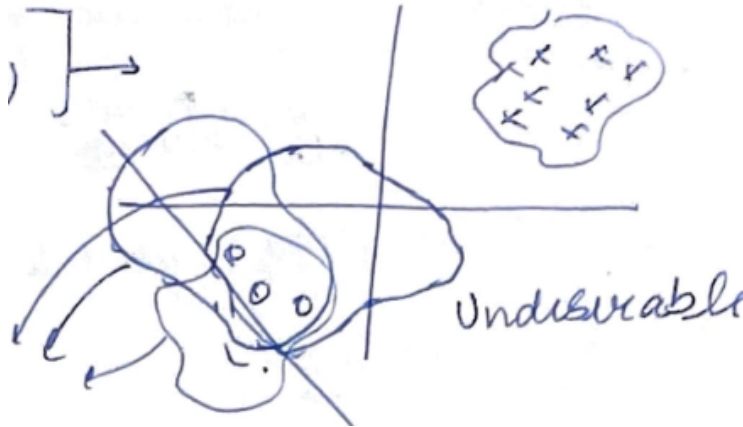
- **Inference:**

$$h_K(x) = \arg \max_j \sum_r \alpha_r \mathbf{1}[h_r(x) = j] \rightarrow \text{Weighted majority voting}$$

Class Imbalance Problem

97 – Normal (Benign)

3 – Cancer (Malignant)



Can't tell what the distribution is based on 3 samples.

2 Possible Problems:

1. Training accuracy not going up
2. Very high training accuracy without actually solving the actual problem

Solutions:

1. Sampling based
2. Loss fn. based
3. Generative sample based

Sampling Strategies

1. Fractional Sampling:

Select $x, y \in \mathbb{R}^+$ s.t. $x \cap \omega_1 = y \cap \omega_2$, where $x + y = 1$

$$\frac{x}{y} = \frac{n_{\omega_1}}{n_{\omega_2}} \Rightarrow x = \frac{m \omega_2}{n_{\omega_1} + n_{\omega_2}}, \quad y = \frac{m \omega_1}{n_{\omega_1} + n_{\omega_2}}$$
$$\Rightarrow \boxed{x = \frac{m \omega_2}{N}, \quad y = \frac{m \omega_1}{N}}$$

Eg: $n_{\omega_1} = 50, \quad n_{\omega_2} = 450, \quad x = 0.9, \quad y = 0.1$

45	45
----	----

Good for ensembling but the problem is that we are wasting a lot of data.

ii) Over Sampling:

Bootstrapped Dataset:

Random selection of 450 samples from ω_1 , with replacement.

$$450 = n_{\omega_2}$$

Can this strategy be used to train K weak classifiers?

iii) Under Sampling:

Bootstrapped Dataset:

if $50 = n_{\omega_1}$, only take n_{ω_1} samples from ω_2

Performs better than oversampling in case of Neural Networks.

2. Generative Sampling:

SMOTE Algorithm: Boosting with Synthetic Oversampling

1. Select a sample x_1 from the minority class ω_1
2. Select 1 of the K nearest neighbours (x_2) from ω_1
3. Multiply $x_1 - x_2$ with a random number from $(0, 1)$

*Underlying assumption: Linear Separability

Other Generative Methods:

Data augmentation: Rotation, Flipping, Addition of Noise

On same data

Operations on latent representation (so that the data distribution is not disturbed)

Weight Scaling

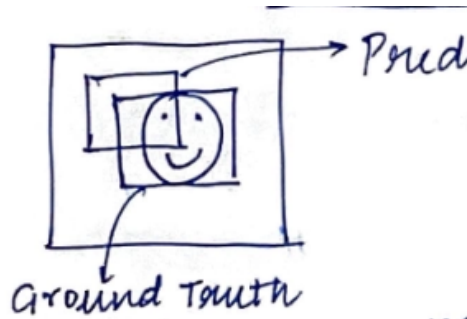
$$\begin{aligned} L &= \frac{1}{2} \sum \|y_i - h(x_i)\|^2 \quad \leftarrow \quad \text{MSE / Binary Cross Entropy / MAE / Weighted BCE} \\ &= \sum_{x_i \in \omega_{\min}} \alpha_{\min} \|y_i - h(x_i)\|^2 + \sum_{x_i \in \omega_{\max}} \alpha_{\max} \|y_i - h(x_i)\|^2 \\ \alpha_{\min} &= \frac{n_{\omega_{\max}}}{N}, \quad \alpha_{\max} = \frac{n_{\omega_{\min}}}{N} \end{aligned}$$

Evaluation Metrics

- Precision (PR) = $\frac{TP}{TP+FP} = \frac{TP}{P'}$
- F1 Score = $\frac{2TP}{P+P'} = \frac{2TP}{2TP+FP+FN}$
- True +ve Rate (TPR) / Recall (R) / Sensitivity: $TPR = \frac{TP}{TP+FN} = \frac{TP}{P}$
- False +ve Rate (FPR): $FPR = \frac{FP}{FP+TN} = \frac{FP}{N}$ $\downarrow$ better
- True -ve Rate (TNR): $TNR = \frac{TN}{FP+TN} = \frac{TN}{N}$ $\uparrow$ better
- False -ve Rate (FNR): $FNR = \frac{FN}{FN+TP} = \frac{FN}{P}$
- $F_{1S} = \frac{2 \times PR \times R}{PR+R}$
- $F_{\beta S} = \frac{(1+\beta^2) \times PR \times R}{\beta^2 \times PR + R} \rightarrow \beta$: To give less or more importance to PR.

*PR & R can be cheated for 100% score, so F1 score can be used as a combined metric.

Intersection over Union (IoU)



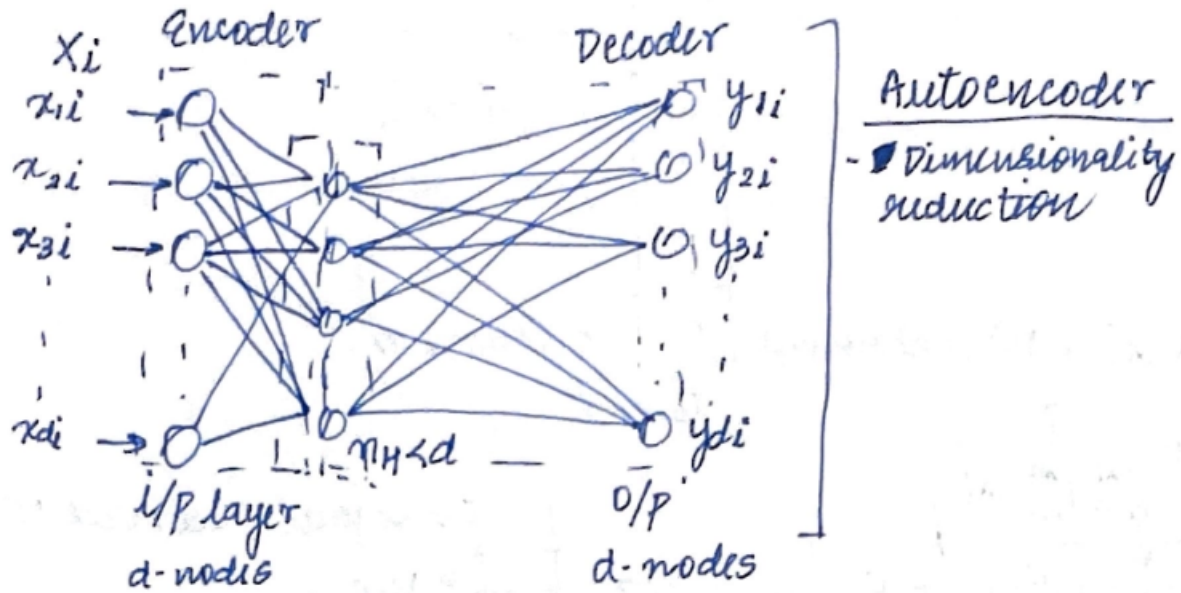
Prediction (P), Ground Truth (GT):

$$\text{IoU} = \frac{GT \cap P}{GT \cup P}$$

Measure of overlap between the 2 bounding boxes.

Prediction is correct if $\text{IoU} \geq \alpha \rightarrow$ Desired Accuracy.

Unsupervised Learning



Loss Fn.:

$$L = \sum_{i=1}^d (y_i - x_i)^2$$

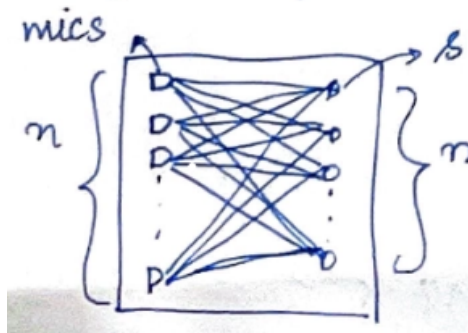
No label, just feature vectors directly compared with o/p.

But y_i is just an approximate of x_i itself.

$\Rightarrow n_H < d$

$\Rightarrow$ Getting a lower dimensional representation of the input.

Independent Component Analysis



Suppose n speakers are simultaneously speaking.

- Each mic records a unique superposition of the voice signals of the speakers.

Goal: Separate out the n signals.

Assume $S \in \mathbb{R}^n$

$$S = \begin{bmatrix} 0 \\ 0 \\ \vdots \\ 1 \\ 0 \end{bmatrix} \rightarrow \text{Source 1, Source 2, } \dots, \text{Source } n$$

Mixing M^x (unknown):

$$X = AS$$

Observation at time t : signals observed by mics 1, 2, $\dots$, n .

Data Available:

$$\left\{ X^{(i)} \mid i = 1, 2, \dots, m \right\}$$

Goal: Find $S^{(i)}$

- $s_j^{(i)}$: sound generated by source j at time i

- $x_j^{(i)}$: signal received by mic j at time i

Let $W = A^{-1}$ (unmixing matrix), full rank M^x :

$$W = \begin{bmatrix} W_1^T \\ W_2^T \\ \vdots \\ W_n^T \end{bmatrix}$$

$$j^{th} \text{ source at time } i: s_j^{(i)} = W_j^T X^{(i)}$$

Some Ambiguities about the Problem

1. Ill-posed problem:

Unique solutions are not always possible.

Take 2 sources S_1, S_2 :

$$\underbrace{\begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix}}_A \underbrace{\begin{bmatrix} S_1^{(i)} \\ S_2^{(i)} \end{bmatrix}}_S = \begin{bmatrix} 2S_1^{(i)} + S_2^{(i)} \\ S_1^{(i)} + S_2^{(i)} \end{bmatrix} = \underbrace{\begin{bmatrix} 1 & 2 \\ 1 & 1 \end{bmatrix}}_{A'} \underbrace{\begin{bmatrix} S_2^{(i)} \\ S_1^{(i)} \end{bmatrix}}_{S'}$$

→ Order of speakers is not defined $\Rightarrow$ can be variable.

→ A' (mixing M^x) is a permutation of $A \Rightarrow A$ is permutation invariant.

$\Rightarrow$ Unique solution does not exist, and many solutions can be correct.

2. Scaling:

If a single column of A is scaled by a factor k (const) and the corresponding source is scaled by $1/k$, there is no way to detect it.

Why is this a problem?

$$A = \begin{bmatrix} 2 & 1 \\ 1 & 1 \end{bmatrix} \quad \begin{bmatrix} S_1^{(i)} \\ S_2^{(i)} \end{bmatrix} = \begin{bmatrix} 2S_1^{(i)} + S_2^{(i)} \\ S_1^{(i)} + S_2^{(i)} \end{bmatrix}$$
$$A' = \begin{bmatrix} 4 & 1 \\ 2 & 1 \end{bmatrix} \quad \begin{bmatrix} S_1^{(i)}/2 \\ S_2^{(i)} \end{bmatrix} = \begin{bmatrix} 2S_1^{(i)} + S_2^{(i)} \\ S_1^{(i)} + S_2^{(i)} \end{bmatrix}$$

If A' is used for regeneration with $\begin{bmatrix} S_1^{(i)} \\ S_2^{(i)} \end{bmatrix}$, it will boost (scale) one of the signals.

3. Need to assume that sources S_i are not $\mathcal{N}(0, I)$ (not rotationally symmetric).

Example: Take 2 speakers $\mathcal{N}(0, I_{2 \times 2})$

$$x^{(i)} = AS^{(i)}$$

Covariance:

$$E[xx^T] = E[ASS^T A^T] = AA^T$$

Let R be an orthogonal M^x (rotation matrix):

$$\boxed{RR^T = R^T R = I}$$

Assume mixing M^x $A' = AR$:

$$E[xx^T] = E[A'SS^T(A')^T] = A'(A')^T = ARR^T A^T = AA^T$$

$$x, x' \sim \mathcal{N}(0, AA^T) \Rightarrow \text{Cannot determine whether we are working with } A \text{ or } AR$$

because x and x' , even though different, belong to the same distribution.

Effect of Linear Transforms on Densities

Suppose we have a random variable $S \sim p_S(S)$, $S \in \mathbb{R}$.

$$X = AS \Rightarrow W = A^{-1}$$

$$\text{Example: } S \sim U[0, 1], x = 2S \Rightarrow W = \frac{1}{2}$$

$$p_X(x) = 0.5 U[0, 2]$$

Multivariate:

$$p_X(x) = p_S(Wx) |W| = p_S(S) |W|$$

ICA Algorithm

Assumptions: sources are i.i.d. (non-Gaussian)

Joint Distribution:

$$P(S) = \prod_{i=1}^n p_S(s_i)$$
$$\Rightarrow p(x) = \prod_{i=1}^n p_S(W_i^T x) |W|$$

Training Samples: $X^{(i)}, i = 1, 2, \dots, m$

$$\ell(W) = \sum_{i=1}^m \left(\sum_{j=1}^n \log p_S(W_j^T x) + \log |W| \right)$$

Maximize iteratively:

$$W_i := W + \Delta W, \quad \alpha \frac{\partial J}{\partial W} = \frac{\partial \ell(W)}{\partial W}$$

Gradient:

$$\nabla_W |W| = |W| (W^{-1})^T \Rightarrow \nabla_W \log |W| = (W^{-1})^T$$
$$\frac{\partial \ell}{\partial W_{k\ell}} = m (W^{-1})_{k\ell}^T + \sum_{i=1}^m \frac{p'_S(W_k^T x)}{p_S(W_k^T x)} x_{\ell}$$

$$\Delta W \propto [W^{-1}]^T + \frac{1}{m} \sum_{i=1}^m g(W x^{(i)}) x^{(i)T}$$

Convolution

Suppose you have a measurement or reading that you're not very sure about:

→ A way to remeasure would be by taking a weighted sum of all the entries before & including the current one, where you give more weight to the newer entries.

→ What you're basically doing is re-evaluating an entry based on its neighbours, and it's based on the premise that the closer the neighbours, the greater the similarity.

Eg: Re-evaluating a pixel on an image based on its neighbours using a kernel with weights assigned as per requirement:

↪ Eg:

$$\begin{bmatrix} 0 & -1 & 0 \\ -1 & 5 & -1 \\ 0 & -1 & 0 \end{bmatrix}$$

: Sharpens the pixels

→ 5: Enhance current pixel

→ -1's: Subtract elements of neighbours from current pixel

Machine Learning

- Manually crafting the convolutional kernels for extracting different types of features
- Feature Engineering

Deep Learning

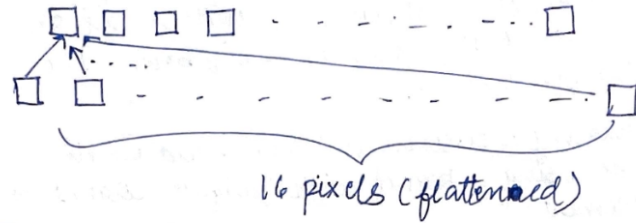
- Learning these kernels as part of the learning problem.

Difference b/w ANN & CNN

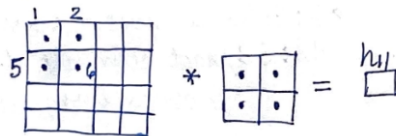
Take the problem of handwritten digit classification:

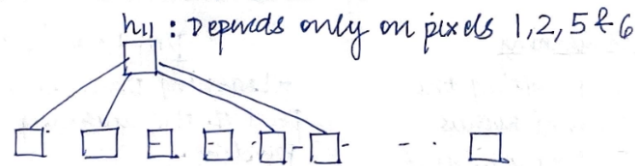
1. Sparse Connections

ANN:  10 classes



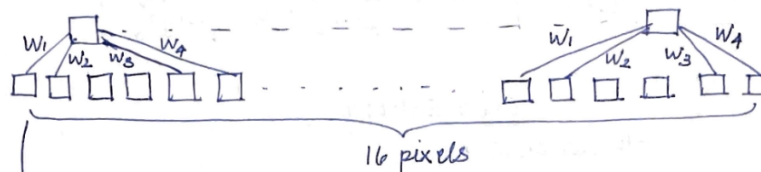
- Each neuron's output depends on every input at every layer

CNN: 



- We are able to have sparser connections by exploiting the fact that the pixels that are further away from each other have little to no connection. This is why we use a kernel, so that pixels are re-evaluated in close neighbourhoods only.

2. Weight Sharing — The kernel applied for one convolution operation remains unchanged throughout the operation, because we want to see the effect of just 1 kernel per operation, or want a specific feature extracted as opposed to a mix of features.



↔ Just 4 weights in layer 1: Underfit? → use multiple kernels in each layer.

3. Shift-Equivariance — Extraction of only features using kernels. CNNs are able to focus on actual features irrespective of their location.

* Even sized filters: Decimal Padding Size — so only odd sized kernels are used.

Kernel Size

Small:

- Fewer Params: Complexity ↓, Memory Req. ↓
- Better at learning local features such as edges, corners, and textures etc.
- Deeper Architecture: more abstract & hierarchical representations

Large:

- Larger receptive field; allows to capture more global info
- Better spatial equivariance
- Computational Efficiency: Required # of convolutional operations ↓

Softmax & Cross Entropy

Softmax squashes the outputs so that the final outputs are probabilistic:

$$S(y_i) = \frac{e^{y_i}}{\sum e^{y_j}}$$

$$D(y, \hat{y}) = - \sum y_i \log_e(\hat{y}_i)$$

Eg: $y = [1, 0, 0]$

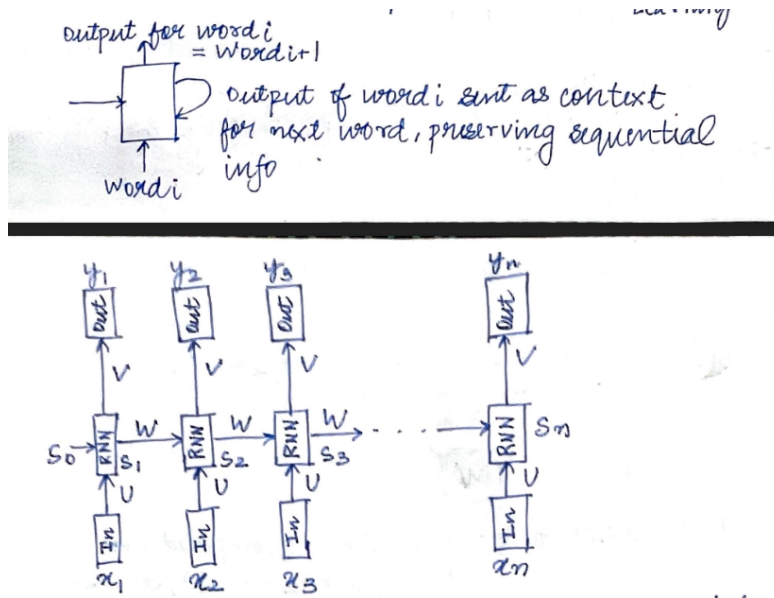
$\hat{y}_1 = [0.7, 0.2, 0.1]$, $\hat{y}_2 = [0.1, 0.3, 0.6]$

$D_1 = 0.35 < D_2 = 2.30 \Rightarrow$ Better model

Recurrent Neural Networks

NNs that can feed the output at time t_1 to the input at time t_2 (Feedback Mechanism).

Eg: In a word prediction problem where the subsequent words are dependent on the previous words (sequential learning).



$$s_i = \sigma(Ux_i + Ws_{i-1} + b)$$

$$y_i = O(Vs_i + c)$$

→ The same function gets executed at every time step, which ensures that the # of params is independent of the length of the input.

- s_i is the state of the network at time step i : it encodes everything that has happened so far.

Backpropagation through Time

$$\text{Total Loss (LL}(\theta)) = \sum_{t=1}^T L_t(\theta)$$

(summation of losses at all time steps)

For backpropagation, we need to compute gradients w.r.t W, U, V :

1.

$$\frac{\partial LL(\theta)}{\partial V} = \sum_{t=1}^T \frac{\partial L_t(\theta)}{\partial V}$$

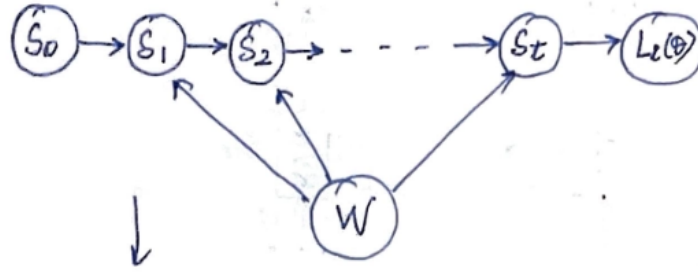
Derivative of loss w.r.t the weights in the output layer.

2.

$$\frac{\partial LL(\theta)}{\partial W} = \sum_{t=1}^T \frac{\partial L_t(\theta)}{\partial W}$$

Obtained by summing gradients along all paths from $L_t(\theta)$ to W .

Paths connecting $L_t(\theta)$ to W



Ordered Network: Each state is computed one at a time ($s_1 \rightarrow s_2 \rightarrow \dots$), because subsequent states depend on previous ones.

Eg:

$$\frac{\partial L_4(\theta)}{\partial W} = \frac{\partial L_4(\theta)}{\partial s_4} \frac{\partial s_4}{\partial W} \quad (\text{Simple: Direct Dependency})$$

$$\frac{\partial s_4}{\partial W} = \frac{\partial}{\partial W} [\sigma(W s_3 + b)] \quad (\text{again a fn of } W)$$

Explicit: $\frac{\partial^+ s_4}{\partial W}$, treating all other inputs as constant

+ Implicit: Summing over all *indirect* paths from s_4 to W .

$$\frac{\partial s_4}{\partial W} = \frac{\partial^+ s_4}{\partial W} + \frac{\partial s_4}{\partial s_3} \frac{\partial s_3}{\partial W}$$

$$\Rightarrow \frac{\partial s_t}{\partial W} = \sum_{k=1}^t \frac{\partial s_t}{\partial s_k} \frac{\partial^+ s_k}{\partial W}$$

Gradients w.r.t V, W & U

$$1. \quad \frac{\partial L(\theta)}{\partial V} = \sum_{t=1}^T \frac{\partial L_t(\theta)}{\partial V}$$

$$2. \quad \frac{\partial L(\theta)}{\partial W} = \sum_{t=1}^T \left(\frac{\partial L_t(\theta)}{\partial s_t} \cdot \sum_{k=1}^t \frac{\partial s_t}{\partial s_k} \frac{\partial^+ s_k}{\partial W} \right)$$

$$3. \quad \frac{\partial L(\theta)}{\partial U} = \sum_{t=1}^T \left(\frac{\partial L_t(\theta)}{\partial s_t} \cdot \sum_{k=1}^t \frac{\partial s_t}{\partial s_k} \frac{\partial^+ s_k}{\partial U} \right)$$

Exploding & Vanishing Gradients

$$\frac{\partial s_t}{\partial s_k} = \prod_{j=k}^{t-1} \frac{\partial s_{j+1}}{\partial s_j}$$

Let $a_j = W s_{j-1} + b$ (pre-activation), $s_j = \sigma(a_j)$. Then

$$\frac{\partial s_j}{\partial s_{j-1}} = \frac{\partial s_j}{\partial a_j} \frac{\partial a_j}{\partial s_{j-1}} = \text{diag}(\sigma'(a_j)) W$$

$$\sigma'(a_j) \leq \frac{1}{4} = \gamma \text{ (logit)}, \quad \sigma'(a_j) \leq 1 = \gamma \text{ (tanh)}$$

$$\left\| \frac{\partial s_j}{\partial s_{j-1}} \right\| \leq \left\| \text{diag}(\sigma'(a_j)) \right\| \|W\|$$

$$\begin{aligned} \Rightarrow \left\| \frac{\partial s_j}{\partial s_{j-1}} \right\| &\leq \gamma \|W\| \\ \Rightarrow \left\| \frac{\partial s_j}{\partial s_{j-1}} \right\| &\leq \gamma \lambda \\ \rightarrow \left\| \frac{\partial s_j}{\partial s_{j-1}} \right\| &= \left\| \prod_{j=k+1}^t \frac{\partial s_j}{\partial s_{j-1}} \right\| \\ \Rightarrow \left\| \frac{\partial s_t}{\partial s_k} \right\| &\leq (\sigma \lambda)^{t-k}, \quad \lambda = \|W\| \end{aligned}$$

- If $\sigma \lambda < 1$: Gradient vanishes $\Rightarrow$ No more updates.
- If $\sigma \lambda > 1$: Gradient explodes $\Rightarrow$ Weights oscillate.

Quick Fixes:

1. Truncated Backpropagation
2. Normalization
3. Use ReLU

Long Short Term Memory Cells (LSTMs)

- Uses different paths for long term and short term memory as feedback for new prediction.
- LSTMs use sigmoid & tanh activation fns.
- At time step t , there is a hidden state h_t & a cell state c_t . The cell state stores long-term information; the LSTM can read, write and erase info from the cell.
- The selection of which info is read/written/erased is controlled by 3 (corresponding) dynamic gates whose values are computed based on current context.

1. **Forget Gate** (what is kept vs forgotten)

$$f_t = \sigma(W_f [h_{t-1}, x_t] + b_f)$$

2. **Input Gate** (what parts of new content are written)

$$i_t = \sigma(W_i [h_{t-1}, x_t] + b_i)$$

3. **Output Gate** (what parts of the cell go to hidden state)

$$o_t = \sigma(W_o [h_{t-1}, x_t] + b_o)$$

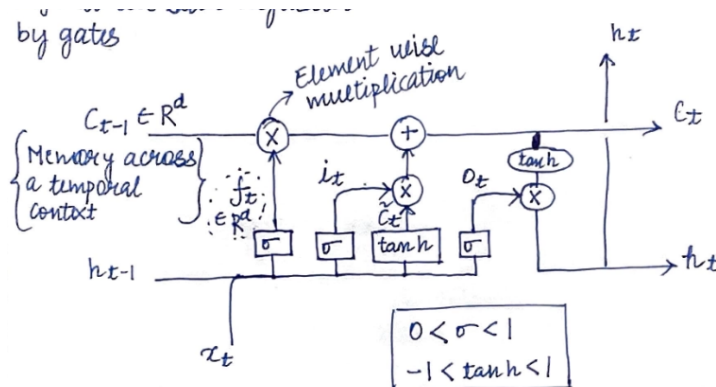
New Cell Content:

$$\tilde{C}_t = \tanh(W_c [h_{t-1}, x_t] + b_c)$$

Cell State & Hidden State:

$$c_t = f_t * c_{t-1} + i_t * \tilde{C}_t \quad h_t = o_t * \tanh(c_t)$$

- Ability to remove or add info to cell state is regulated by gates.



1. f_t : controls what is kept vs what is forgotten from previous cell state ($0 < \sigma < 1$ for each dim).
2. i_t : decides what information to throw away / add to the cell state (context-driven).
3. $\tilde{C}_t$: new content to be written to cell.

4. c_t (final cell state) combines “erase” via $f_t * c_{t-1}$ and “write” via $i_t * \tilde{C}_t$ (both based on context).
5. o_t : controls what parts of cell are output to hidden state.
6. h_t : reads some content from cell to provide output at current timestep as well as context for next step.

LSTM: Solution to Vanishing Gradient Problem

- Gradient Highway.
- Gradient (cell memory) at c_{t-1} is passed to c_t unaffected by other operations except the forget gate.
- The forget gate is a vector (not a layer with weights); it reduces the gradient where it should, based on context.

Detecting Context Change

Question. How do we know that the context has changed?

Answer. Take 3 words in the n -dimensional space: *king*, *queen*, *dog*.

- *king* & *queen* are close (similar); *dog* is far (dissimilar).
- Their vector representations reflect this: similar for *king* & *queen*, very different from *dog*.

Now calculate similarity / dissimilarity:

1. Cosine similarity:

$$\cos \theta = \frac{A \cdot B}{\|A\| \|B\|}$$

2. Sigmoid (Normalised C): < 0.5 : Context Unchanged > 0.5 : Context Changed.

Word Representations

- **Corpus:** The complete set of sentences
- **Vocabulary (V):** The set of all distinct words in the corpus

One Hot Representation (Sparse)

$$\text{Word}_i : [0 \ 0 \ \dots \ 1 \ 0 \ \dots \ 0]$$

→ index i of vector of size V

Drawbacks:

1. V tends to be very large
2. No notion of similarity (Euclid-dist = $\sqrt{2}$ for word pairs)

Continuous Bag of Words Model (CBOW) & Skip Grams (SG)

Distributional Semantics: A word’s meaning is given by the words that frequently appear close-by.

When a word w appears in a text, its context is the set of words that appear within a fixed-size window, and we use them to build a representation of w .

Word Vectors: We will build a dense vector for each word, chosen so that it is similar to vectors of words that appear in similar contexts, measuring similarity as the dot product of vectors.

1. Skip Grams – Predict context words (position independent) given center word
2. Continuous Bag of Words – Predict center word from (bag of) context words

Word2Vec

Given a large corpus of text: Each word in a fixed vocabulary is represented by a vector.

Idea:

1. Go through each position t in the text, with center word c & context words o
2. Use the similarity of vectors for c & o to calculate $P(o|c)$ or vice versa (CBOW, SG)
3. Keep adjusting the word vectors to maximize probability
Probability of occurrence of context words (words that appeared with c) should be high

Objective Function:

For each position $t = 1, \dots, T$, predict context words within a window of fixed-size (m), given center word w_t :

$$\text{Likelihood } L(\theta) = \prod_{t=1}^T \prod_{-m \leq j \leq m} P(W_{t+j} | W_t; \theta)$$

Cost (objective) FN = -ve avg of log likelihood:

$$J(\theta) = -\frac{1}{T} \log L(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m} \log P(W_{t+j} | W_t; \theta)$$

Minimizing $J(\theta) \Leftrightarrow$ Maximizing Predictive Accuracy.

$$P(o|c) = \frac{\exp(U_o^T V_c)}{\sum_{w \in V} \exp(U_w^T V_c)} \rightarrow \text{Dot product compares similarity of } o \text{ \& } c; \text{ larger dot product} \Rightarrow \text{larger probability.}$$

* Normalization over entire vocab to give probability distribution.

* Two vectors per word w : V_w : w is center word; U_w : w is context word.

Training Word2Vec

$$\theta = [V_{w_1}, U_{w_1}, V_{w_2}, U_{w_2}, \dots, U_{w_V}]^T \in \mathbb{R}^{2dV}$$

Represents all model params in one long vector.

Optimization through gradient descent:

$$J(\theta) = -\frac{1}{T} \sum_{t=1}^T \sum_{-m \leq j \leq m} \log P(W_{t+j} | W_t)$$
$$P(o|c) = \frac{\exp(U_o^T V_c)}{\sum_{w \in V} \exp(U_w^T V_c)}$$
$$\frac{\partial}{\partial V_c} \log P(o|c) = U_o - \sum_{x \in V} P(X|c) U_x$$

Initialization of the word vectors is done in the same way as model weights: randomly initialize close to 0.

Encoder-Decoder Models

Sequence learning problem:

1. Data: All sentences from a large corpus
2. Model: $s_t = \sigma(W s_{t-1} + U z_t + b)$ (for RNN)
 $P(y_t = j | y^{t-1}) = \text{softmax}(V s_t + c)_j$
3. Parameters: U, V, W, b, c
4. Loss: $L(\theta) = \sum_{t=1}^T L_t(\theta)$
 $L_t(\theta) = -\log P(y_t = e_t | y^{t-1})$ (true word)
5. Algorithm: BPTT

For image captioning, we need another input for the model, which is the image itself:

$$P(y_t | y_1^{t-1}, \mathcal{I}) \rightarrow \text{Entire context from } y_1 \text{ to } y_{t-1}.$$

$$P(y_t | y_1^{t-1}) = P(y_t | s_t), \quad P(y_t | y_1^{t-1}, \mathcal{I}) = P(y_t | s_t, f_{c7}(\mathcal{I}))$$

All info in the context is passed via s_t ; all info in the image via $f_{c7}(\mathcal{I})$ (fully connected layer of CNN).

How to Model This?

Option 1

- $s_0 = f_{c7}(\mathcal{I})$
- Now all subsequent s_t 's also depend on $f_{c7}(\mathcal{I})$
- $\Rightarrow P(y_t = j)$ depends on s_t and we compute $P(y_t = j \mid s_t, f_{c7}(\mathcal{I}))$.

Option 2

- $s_t = \text{RNN}(s_{t-1}, [x_t, f_{c7}(\mathcal{I})])$
- We are explicitly using $f_{c7}(\mathcal{I})$ to compute s_t (instead of implicitly), so: $P(y_t = j)$.
- $\Rightarrow$ A CNN is used to *encode* the image and an RNN is then used to *decode* (generate) a sentence from the encoding.
- * The models used for the encoder & decoder depend on the task at hand (e.g., for image generation from text, the encoder may be an RNN and the decoder a generative model like a GAN).

Attention Mechanism

- For predicting the output at timestep t , is each and every input even important?
- For context, can we instead pass a weighted sum of the inputs, with more weight attached to more important inputs?
- How do we know which inputs are important at which timestep?
 - We don't, but it can be learned from the data.

$$e_{jt} = f_{\text{ATT}}(s_{t-1}, h_j)$$

At the t^{th} time-step of the decoder, how important is the j^{th} word?

It should depend on that word itself, along with what has happened in the decoder so far.

Normalizing Attention Weights

$$\alpha_{jt} = \frac{\exp(e_{jt})}{\sum_{j=1}^M \exp(e_{jt})}$$

Probability of focusing on the j^{th} word to produce the t^{th} output word.

Question. But the true values of α_{jt} are not even known to us; why does this model work better?

Answer. Because it is an addition of parameters on an already great model; it is a more informed way of learning, and given enough data, it should be able to learn these attention weights like humans do (reinforcement-type learning).

Evaluation of Language Model: Perplexity

- A model that assigns a higher probability to real or frequently observed sentences (rather than rarely observed or ungrammatical ones) is a better model.
- The best language model is one that best predicts an unseen test set.
- Given the solution to a language modeling task is the sentence s , the best model is the one which gives the highest probability for s compared to all models.
- Perplexity is the probability of the test set, normalized by the # of words.

$$\text{PP}(S) = P(w_1, w_2, \dots, w_N)^{-1/N} = \left(\prod_{i=1}^N P(w_i \mid w_1, \dots, w_{i-1}) \right)^{-1/N}$$

Lower Perplexity $\Rightarrow$ Better LM.

Attention Output from Probability Distribution (α_t)

At timestep t , the probability distribution is

$$\alpha_{jt} = \frac{\exp(e_{jt})}{\sum_{k=1}^M \exp(e_{kt})} \quad \Rightarrow \quad \alpha_t = [\alpha_{1t} \ \alpha_{2t} \ \dots \ \alpha_{Mt}]$$

Now use α_t to take a weighted sum of the encoder's hidden states to get the attention output

$$a_t = \sum_{j=1}^M \alpha_{jt} h_j \in \mathbb{R}^h$$

Finally, concatenate the attention output a_t with the decoder's hidden state s_t & proceed as in the non-attention seq2seq model:

$$[a_t; s_t] \in \mathbb{R}^{2h}$$

Variations in Attention

- We have values $h_1, \dots, h_N \in \mathbb{R}^d$ and a query $s_t \in \mathbb{R}^{d_2}$.
- Attention always involves:
 1. Computing the attention scores $e \in \mathbb{R}^N$ (multiple ways $\Rightarrow$ variations),
 2. Taking softmax for attention distribution $\alpha \in \mathbb{R}^N$,
 3. Using α to take a weighted sum of values and get attention output a .

Positional Encoding (Transformers)

$\text{PE}_{\text{pos}, 2i} = \sin\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right), \quad \text{PE}_{\text{pos}, 2i+1} = \cos\left(\frac{\text{pos}}{10000^{\frac{2i}{d_{\text{model}}}}}\right)$
--